Here is the formal write-up on the topic 'Web Technology' in Markdown format:

Web Technology

1. HTML: HTML stands for Hypertext Markup Language. It is the main markup language used to create web pages. HTML uses a series of elements to structure a web page semantically. Some important HTML elements are

   ,

   ,

   ,

   ,

   ,

   , , , etc.

2. CSS: CSS stands for Cascading Style Sheets. It is used to style an HTML document and define colors, fonts, layout, etc. CSS is used to separate the presentation of a document from its structure (specified in HTML). Some common ways to include CSS in an HTML document are - inline styles,

   element in the HTML head, and external .css files.

3. JavaScript: JavaScript is a lightweight, interpreted programming language typically embedded into web pages. It allows web pages to be interactive and dynamic. It can be used for client-side scripting to interact with the user, control the browser, modify CSS styles, create animations, validate forms, and much more. JavaScript code is executed by the browser and runs locally on the client machine.

4. Web APIs: Web APIs refer to the sets of requirements that govern the interactions between a web application and a web browser. They allow web applications to interact with the browser and operating system. Some important web APIs are DOM, browser history, geolocation, web storage, web workers, web sockets, etc. These APIs enable web developers to create dynamic and interactive web applications.

The above points cover the key aspects of web technology in a formal and concise manner without any emojis or external links as required. Please let me know if you would like me to elaborate on any of the points or modify the content in any way.

Here is the formal content in Markdown format without any emojis or external links:

Unit 1 - Introduction to Web Technology

1. Introduction to the Internet

- The Internet is a global network of computers and other devices interconnected through various methods of digital communications.
- It is made up of smaller networks that are connected to each other. These networks might be at your home, at your office, or anywhere else.
- Data is transferred between networks and devices using a common set of rules and protocols called TCP/IP protocol suite.

2. Introduction to the World Wide Web

- The World Wide Web or simply web is a massive collection of interconnected web pages that can be accessed through the Internet.
- Web pages are downloaded from web servers onto the user's device using a software application called a web browser.
- Web pages are written in HTML (Hypertext Markup Language) and can contain multimedia content like images, videos, etc.
- The web has become an important part of how we access and share information in today's world.

3. How the Web Works

- The working of the web involves web servers, web browsers, and the HTTP protocol.
- When a user enters a URL into the browser, an HTTP command is sent to the web server directing it to fetch and transmit the requested webpage.
- The web server locates the page and sends it back to the browser as an HTTP response. The browser then displays the webpage on the user's device.
- This whole process happens very quickly and allows us to access web pages seamlessly.

Here is the formal markdown content for the given topic:

Introduction to Web Technology

1. Web: The World Wide Web (WWW) refers to the massive collection of interconnected web pages that can be accessed via the Internet. The web pages are written using Hypertext Markup Language (HTML) and identified by Uniform Resource Locators (URLs).

2. HTTP: Hypertext Transfer Protocol (HTTP) is the set of rules for transferring web pages on the World Wide Web. It defines how messages should be formatted and transmitted, and what actions web servers and browsers should take in response to various commands.

3. HTML: Hypertext Markup Language (HTML) is the main markup language used to create web pages. It uses a series of elements consisting of

tags enclosed in angle brackets. HTML tags tell the browser what type of content is being presented.

4. URL: A Uniform Resource Locator (URL) is a reference to a web resource that specifies its location on a computer network and a mechanism for retrieving it. A URL is composed of different parts including the scheme name (such as http) followed by a colon and two forward slashes, then the domain name, and path to the resource.

5. Web Browser: A web browser is a software application used to locate, retrieve and display web pages. Popular web browsers include Chrome, Firefox, Safari, Internet Explorer, and Microsoft Edge. The browser retrieves the HTML document for a web page and renders it on the user's device.

Does this look okay? Let me know if you would like me to modify or add anything.

Here is the content in markdown format with formal tone and without any external links or emojis:

Web Development Strategies

1. Progressive Enhancement: This strategy involves developing a basic web page that works on all browsers and devices, then enhancing the experience for more modern browsers and devices with additional features. This ensures maximum accessibility and compatibility.

2. Mobile First: This strategy involves designing a website for mobile devices first, then scaling up the design for larger screens. Since mobile devices are increasingly the primary way people access the web, this ensures the website is optimized for the most common user experience.

3. Responsive Design: With this strategy, the website is designed to dynamically adapt to different screen sizes and devices. The layout changes based on the user's viewport to provide an optimized experience on any device. This is achieved using flexible grids, images, and components.

4. Server Side Rendering: With this strategy, the initial HTML is rendered on the server and sent to the browser, then the client side app takes over. This leads to faster load times and better SEO since the crawlers get full HTML pages. However, it can be more complex to implement and host.

5. Static Site Generation: This strategy pre-renders HTML pages at build time. The entire site can be hosted as flat HTML files, leading to very fast load times and good performance. However, the content must be pre-generated and cannot be dynamic. It suits sites with mainly static content.

Here is the content in formal tone without emojis or external links written in Markdown format with headings:

History of Web

1. 1989 - Tim Berners-Lee invented the World Wide Web. He wrote the first web browser computer program in 1989 while at CERN in Switzerland.
2. 1990 - Tim Berners-Lee created the first web server computer program and the first web pages, which described the project itself. The world's first web browser was released in 1990 for mainframe computers.
3. 1993 - The first commercial web browser Mosaic was released. Mosaic was a graphical browser developed at National Center for Supercomputing Applications (NCSA). It helped popularize the web and played a major role in the rise of the Internet as a mass-market medium.
4. 1994 - The first commercial web browser Netscape Navigator was released by Netscape Communications Corporation. It was a more advanced browser than Mosaic with features like cookies and JavaScript support which accelerated the popularity of the web.
5. 1998 - The open source browser Mozilla Firefox was released. It was based on Netscape's browser. Google Chrome was also released in 2008 which added competition to the browser market.
6. Today - The web has become a massive network of interconnected websites and web-based services used by billions of people around the world. Websites have become more complex with features like ecommerce, social media, video and audio streaming, and more. Modern browsers have also become more advanced, secure and powerful.

The content is written in points in a formal tone with no emojis or external links as specified. Please let me know if you would like me to modify or expand the content.

Here is the content in formal tone with markdown formatting:

History of Internet

1. ARPANET: The first network that implemented TCP/IP protocol suite was ARPANET (Advanced Research Projects Agency Network) funded by U.S. Government's Advanced Research Projects Agency (ARPA) in late 1960s and early 1970s. It connected major universities and research centers. Email and FTP were the major applications used.
2. NSFNET: In 1980s, the National Science Foundation (NSF) funded NSFNET project to provide networking facility to scientists and researchers. It replaced ARPANET in 1995.
3. Commercialization: In mid-1990s, the US government ended funding to NSFNET and the control went to networking companies. This led to widespread commercialization of the Internet. This resulted in the growth of the World Wide Web, web browsers, and web services.

4. Growth and global connectivity: In 1990s, the growth of the Internet was tremendous due to web, e-commerce, and social networking applications. This growth has continued exponentially and resulted in global connectivity and access for billions of users worldwide.
5. New applications: The Internet has enabled new applications like social media, streaming media, Voice over Internet Protocol (VoIP), Internet of Things (IoT), and more. This has transformed numerous business and social interactions and activities.

The above points cover the key milestones in the history of the Internet in a formal tone with markdown formatting and without any emojis or external links. Please let me know if you would like me to elaborate on any of the points or modify the content.

Here is the formal content on the topic "Protocols Governing Web" in markdown format without any feeling or friendliness:

Protocols Governing Web

1. Hypertext Transfer Protocol (HTTP): HTTP is the fundamental communication language of the World Wide Web. It defines how messages should be formatted and transmitted, and what actions web servers and browsers should take in response to various commands.
2. Hypertext Markup Language (HTML): HTML is the main markup language for creating web pages and web applications. It uses a series of elements consisting of tags enclosed in angle brackets. HTML tags most commonly come in pairs like

   and

   , with the first tag in a pair opening a element and the second tag closing it.
3. Cascading Style Sheets (CSS): CSS is a style sheet language used for describing the presentation of a document written in a markup language like HTML. CSS is designed primarily to enable the separation of document structure from document presentation, including elements such as the layout, colors, and fonts.
4. Uniform Resource Locators (URLs): URLs are strings of characters that specify the location of a resource on the Internet. They are fundamental to the operation of the Web, and are used to locate and retrieve a web page, image, video, or other resource. A URL typically consists of several parts including the scheme, host, path, and query.
5. The TCP/IP Protocol: The TCP/IP protocol is a set of rules for transferring data between computers on the Internet. TCP handles transmission control, breaking data into packets that are reassembled at the receiver's

end. IP handles addressing, ensuring that data is delivered to the correct recipient. TCP/IP is fundamental to communication on the Internet.

Here is the formal content in Markdown format without any emojis or external links:

Writing Web Projects

1. Choose a Project Idea

- Select a project idea that interests you and is within your skills level. Some beginner-friendly project ideas include:
  - A basic personal website to share your bio and work
  - A movie or music review website
  - An e-commerce store for a hobby interest (e.g. selling homemade crafts or digital downloads)

2. Plan the Project Scope and Features

- Determine the key pages and core features your project will include. Figure out the minimum viable product and nice-to-have additional features. Map out the information architecture and user flow through the site.

3. Choose Your Tech Stack

- Select the programming languages, frameworks, and libraries you will use to build the project. Some common stacks for web projects include:
  - HTML, CSS, and JavaScript
  - Python and Django/Flask
  - Ruby on Rails
  - PHP and a CMS like WordPress

4. Design the User Interface

- Sketch wireframes and mockups for your site layout and key pages. Determine the stylistic elements like colors, fonts, images, and other visuals you will include. You can design your own UI or use pre-built templates as a starting point.

5. Implement the Project

- Using your chosen tech stack, implement your website based on the plans and designs. Build out the pages and features, add content, and handle functionality and interactivity. Test the project and fix any bugs or issues before launching.

6. Launch and Iterate

- Launch your project live on the web. Then monitor user feedback and analytics to determine how to improve the user experience and add additional features. Release regular updates to keep the project evolving and interesting.

Here is the formal content in Markdown format without emojis or external links:

Connecting to Internet

1. Obtain an Internet connection:

- Dial-up: Uses phone line and modem to connect to Internet. Slow speed.
- DSL: Digital Subscriber Line. Uses phone line but separate from voice. Faster than dial-up.
- Cable: Uses cable TV line. Generally faster than DSL.
- Fiber: Connects via fiber optic cable. Fastest speed.
- Mobile data: Uses cellular network. Convenient but slower and data capped.

2. Obtain necessary hardware:

- Router: Connects to Internet source and creates wireless network.
- Modem: Required for dial-up and some DSL/cable connections. Converts data to suitable format.
- Ethernet cables: Required to connect devices to router for wired connections.

3. Configure network settings:

- SSID: Wireless network name.
- Password: Secure wireless network.
- Network type: DHCP (automatic) or manual IP address assignment.
- Firewall: Blocks unauthorized access. Adjust settings as needed.

4. Connect devices:

- Wired: Ethernet cable from device to router.
- Wireless: Connect to Wi-Fi network with network name and password.

5. Test connection:

- Browser: Check that websites can be accessed.
- Speed test: Test download/upload speeds to ensure expected performance.
- Device connectivity: Ensure all devices can access the network and Internet.

Here is the formal content in Markdown format without any emojis or external links:

Introduction to Internet services

1. Email: Email or electronic mail is one of the most popular Internet services. It allows people to exchange messages, photos, files, and more over the Internet. Users can access their email on computers, phones, and tablets using email applications and websites.

2. World Wide Web: The World Wide Web, commonly known as the Web, is a system of interlinked hypertext documents that are accessed via the Internet. The Web allows access to a vast array of information on almost any topic. Users access the Web using a web browser to view web pages, videos, images, applications, and more.
3. File sharing: There are many Internet services that allow users to upload and share files with others. These include services like Dropbox, Google Drive, and OneDrive. Users can share photos, documents, videos, music, and other files. These services typically provide both free and paid storage options with various features.
4. Streaming media: Services like YouTube, Netflix, Hulu, and Spotify allow users to stream videos, music, movies, TV shows, and more over the Internet. Users can watch and listen to a wide range of content on demand via streaming. These services are available on computers as well as phones, tablets, and television streaming devices.
5. Social networking: Popular social networking services like Facebook, Instagram, Twitter, and Snapchat allow people to connect, share status updates, photos, videos, messages, and more. These services enable people to stay up to date on what's happening with friends, family, colleagues, and interests. Billions of people use social networking services to interact and share online.

Does this look okay? Let me know if you would like me to modify or add anything.

Here is the content in formal tone without emojis or external links in Markdown format:

Introduction to Internet tools

1. Web Browser: A web browser is a software application for accessing information on the World Wide Web. It enables users to access websites and view web pages. Popular web browsers include Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, etc.
2. Email: Email or electronic mail allows people to exchange messages over the Internet. Email operates across the world and is a key tool for both personal and business communication. Popular email services include Gmail, Outlook, Yahoo Mail, etc.
3. Search Engines: Search engines are tools to search for information on the Internet. They allow users to enter search terms and retrieve relevant results and links. The most popular search engines are Google, Bing, Baidu, Yahoo, etc.
4. Social Networks: Social networks are online platforms that allow people to connect, share updates, photos, videos, messages, and more. Popular social networks include Facebook, Instagram, Twitter, LinkedIn, Pinterest,

Snapchat, etc.
5. File Sharing: File sharing allows people to upload and share various files like documents, music, videos, software, etc. with others over the Internet. Popular file sharing services include Dropbox, Google Drive, OneDrive, Box, etc.

The content summarizes some of the key Internet tools like web browsers, email, search engines, social networks, and file sharing services in a formal tone with points in Markdown format. Please let me know if you would like me to modify or expand the content in any way.

Here is the content in markdown format with formal tone and without any emojis or external links:

Introduction to client-server computing

1. Client-server computing refers to a distributed application structure that partitions tasks or workloads between the providers of a resource or service, called servers, and service requesters, called clients.
2. Clients are devices or applications that access the service. Servers are computing devices that provide the service. For example, a web browser is a client that accesses the service of a web server.
3. The client-server model distinguishes the client and the server, optimizing for the client (e.g., user interface, usability) and for the server (usually performance, and security). Often the client and server communicate over a computer network on separate hardware, but both client and server may reside in the same system.
4. A primary reason for using a client-server system is that it centralizes management and security in the servers. This model also makes it easy to administer changes and upgrades. The clients are relatively simple in this system and mainly focus on presentation, user interaction, and input/output.
5. Typical advantages of the client-server model are:

- Centralized control and administration.
- More secured from unauthorised access.
- Efficiency through specialized servers.
- Scalability and flexibility. Additional servers can be added as needed.

I hope this helps! Let me know if you would like me to modify or expand the content.

Here is the formal content on Core Java in markdown format without any feeling or friendliness:

Core Java

1. Java Platform: Core Java runs on Java Platform which includes Java Virtual Machine(JVM), Java API and Java Programming Language.
2. Object Oriented: Core Java supports Object Oriented Programming paradigm and revolves around objects and classes. Everything in Java is an Object.
3. Platform Independent: Core Java is platform independent as the bytecode can run on any system having JVM. This makes Java code portable and compatible.
4. Simple: Core Java has a very simple programming language with minimal complexity and easy to read syntax based on C++.
5. Secure: Core Java has a robust and strong security architecture with sandboxed execution of code and no access to system resources.
6. Architecturally Neutral: Core Java compiler generates an architecture-neutral object file format which makes the compiled code executable on many processors.
7. Portable: Being architecture-neutral and having no implementation dependent aspects of the language makes Core Java portable.
8. High Performance: With the use of Just-In-Time compilers, Core Java enables high performance.
9. Multithreaded: Core Java supports multithreaded programming which allows different parts of a program to run concurrently.
10. Dynamic: Core Java is considered to be more dynamic than C or C++ as it supports features like reflection, runtime class loading, etc.

[No external links included as required.]

Here is the content in markdown format with formal tone and without emojis or external links:

Introduction to Java

1. Java is a popular programming language developed by Sun Microsystems in 1991. It is an object-oriented language with a C/C++-like syntax.
2. Java source code is compiled into bytecode which can run on any Java Virtual Machine (JVM). This makes Java code portable and compatible across different platforms.
3. Some key features of Java are:

- Object-oriented: Java supports object-oriented programming concepts like classes, objects, inheritance, encapsulation, etc.
- Platform independent: Java bytecode can run on any system with a JVM.
- Simple: Java has a simple and easy to learn syntax.
- Secure: Java has a secure feature called “sandbox” which isolates untrusted code.
- Multithreaded: Java supports multithreaded programming which allows different parts of a program to run concurrently.

- Architectural-neutral: Java compiler generates an architecture-neutral object file format which makes the compiled code executable on many processors.

4. Some common uses of Java are:

- Mobile app development (Android apps)
- Server-side development (JSP, Servlets, etc.)
- Scientific applications
- Enterprise software
- Gaming
- And more…

Overall, Java is a popular and widely used programming language with a variety of applications in software development. It is easy to learn and runs on multiple platforms making it a practical choice for developers.

Here is the formal content on the topic "Operator in Core Java" in markdown format:

Operator in Core Java

1. Arithmetic Operators: Used for mathematical calculations like addition, subtraction, multiplication, division, modulus, increment, decrement, etc. Eg: +, -, *, /, %, ++, –
2. Relational Operators: Used to check relationships between variables or values. Eg: >, <, >=, <=, ==, !=
3. Logical Operators: Used to combine multiple conditions or to negate a condition. Eg: &&, ||, !
4. Assignment Operators: Used to assign values to variables. Eg: =, +=, -=, *=, /=, %=, <<=, >>=, >>>=, &=, ^=, |=
5. Bitwise Operators: Used to perform bitwise operations on integral types. Eg: &, |, ^, ~, <<, >>, >>>
6. Ternary Operator: A short-hand for writing if-else expression. Takes three operands. Eg: variable = condition ? expression1 : expression2;
7. Type Operators: Used to get the type of a variable or object. Eg: instanceof

The operators can be applied on primitive data types as well as objects. They are a fundamental way to manipulate data and control the flow of a program in Java.

Here is the formal content on the topic 'Data type in Core Java' in markdown format with points:

Data type in Core Java

1. Primitive data types:

- byte: 8-bit signed integer

- short: 16-bit signed integer
- int: 32-bit signed integer
- long: 64-bit signed integer
- float: 32-bit IEEE 754 floating point
- double: 64-bit IEEE 754 floating point
- boolean: true or false
- char: 16-bit Unicode character

2. Wrapper classes:

- Byte, Short, Integer, Long, Float, Double, Boolean, Character
- These are used to convert primitive data types into objects and vice-versa.
- Useful for collections that store objects.

3. String:

- String is a class in Java, not a primitive type.
- It is used to represent textual data.
- Strings are immutable.

4. Casting:

- Widening casting: Converting a smaller type to a larger type
- Narrowing casting: Converting a larger type to a smaller type
- Need explicit casting for narrowing conversions
- Can lose precision if values are incompatible

5. Variable declaration:

- Type name; // Declaration
- Type name = value; // Declaration + initialization

This is the content written in formal tone with points and in markdown format as per the given instructions. No emojis or external links have been used.

Here is the formal content on the topic 'Variable in Core Java' in markdown format:

Variable in Core Java

1. A variable is a location/container which holds the value that can change.
2. In Java, there are three types of variables:
    - Local Variable: Declared inside the body of a method and usable only within that method.
    - Instance Variable: Declared inside a class but outside any method and are instantiated when an object of the class is created. They are shared by all instances (objects) of the class.
    - Static Variable: Also declared inside a class but with the static keyword. There is only one copy of a static variable shared by all instances of the class.

3. Variables must be declared before use. The declaration specifies the data type and name of the variable. For example: int count;
4. Values are assigned to variables using the assignment operator (=). For example: count = 5;
5. The valid names for variables are identifiers which must follow certain rules:
    - The first character must be a letter, underscore or dollar symbol.
    - Subsequent characters can be letters, digits or underscores.
    - Case sensitive: age and Age are different variables.
    - Cannot be a keyword (like int, float, etc.).
6. Data types in Java specify the size and type of values that can be stored in a variable. The most commonly used data types are:
    - int: For whole numbers
    - float: For fractional numbers
    - double: For fractional numbers with greater precision
    - char: For single character
    - boolean: For true/false values

Here is the formal content on the topic "Arrays in Core Java" in markdown format without any emojis or external links:

Arrays in Core Java

1. An array is a data structure in Java that contains a fixed number of elements of a similar data type.
2. To declare an array, you specify the type of elements and [] after the type name. For example, int[] age; declares an array of integers.
3. To instantiate an array, you use the new keyword followed by the type name and a number in [] that specifies the number of elements in the array. For example, age = new int[10]; creates an array of 10 integers.
4. You can access array elements using the index of the element in [] after the array name. Indexes start from 0. For example, age[0] refers to the first element and age[9] refers to the last element in the array age.
5. The length property can be used to find the number of elements in an array. For example, age.length will return 10 for the array age.
6. Multidimensional arrays can also be defined in Java, for example, int[][] multi = new int[3][4]; creates a multidimensional array with 3 arrays of 4 integers each.
7. Arrays can be iterated using for loops and enhanced for loops. The elements can be sorted or searched based on the array type. Various utility methods are available in the Arrays class to manipulate arrays.

The above points cover the key concepts related to arrays in Core Java. Let me know if you would like me to elaborate on any of the points or explain anything else related to arrays.

Here is the content in Markdown format without any emojis or external links,

written in a formal tone with points:

Methods & Classes in Core Java

1. Methods

- A block of code that is executed when it is called
- Defines input (parameters) and output (return type)
- Used to structure programs and increase reusability
- Types: void (no return), int, double, boolean, etc.

2. Classes

- Blueprint for objects that defines data and behavior
- Data: Fields (variables), state (values of fields)
- Behavior: Methods (functions)
- Use access modifiers: public, private, protected to control access
- Fields and methods can have modifiers like final or static
- Objects are instances of a class

3. Packages

- Organize classes into namespaces using packages
- Follow directory structure (package name = folder path)
- Imports are used to access classes from packages
- Example: java.lang, java.util, java.io are standard packages

This content summarizes some key points around methods, classes, and packages in Core Java to be used as study material for learning and exams. The points are written formally without emojis or external links. Please let me know if you would like me to elaborate on any of the points or modify the content in any way.

Here is the formal content on Inheritance in Core Java in markdown format without any emojis or external links:

Inheritance in Core Java

1. Inheritance is an important concept in Object Oriented Programming (OOP). It allows a class to inherit the properties and methods of another class.
2. The class that inherits the properties of another class is called a subclass or child class. The class whose properties are inherited is called a superclass or parent class.
3. Inheritance allows code reusability. The subclass can reuse the code of the superclass, so you don't have to write the same code again.
4. To inherit a class, use the extends keyword. For example, class Bike extends Vehicle {}. Here Bike is the subclass and Vehicle is the superclass.

5. A subclass can have its own methods and properties in addition to the inherited ones. This is called overriding. The subclass can override methods of the superclass if needed.
6. A subclass can be extended further to form a hierarchy of inheriting classes. This is called a class hierarchy. The hierarchy can go up to multiple levels but should not be too deep.
7. The Object class is the root class of all classes in Java. So, ultimately, every class inherits from the Object class.
8. To refer to a superclass constructor, use the super keyword. This is necessary when the subclass constructor needs to call the superclass constructor.
9. An interface can also be extended using the extends keyword. A class can implement multiple interfaces but can extend only one class.
10. Inheritance represents an "is-a" relationship. The subclass "is-a" type of the superclass. For example, a Bike "is-a" Vehicle. Inheritance should only be used if this relationship is true.

Here is the content in markdown format with formal tone and without any emojis or external links:

Package and Interface in Core Java

- Package: A package is a grouping of related types (classes, interfaces, enumerations and annotations) providing access protection and name space management.
- Benefits of using packages: – Avoid naming conflicts: Types with the same name can exist in different packages. – Makes searching/locating and usage of types easier. – Provides access control (public, default, protected, private). – Organizes related types together.
- Creating packages: Packages are created using the `package` keyword as the first statement in a source file. E.g. `package com.example.mypackage;`
- Importing packages: The `import` keyword is used to import types into your source file for use. E.g. `import com.example.mypackage.MyClass;`
- Interface: An interface is a contract between a class and the outside world. It defines what a class must do, but not how. Interfaces cannot be instantiated and can extend multiple interfaces.
- Benefits of interfaces: – Promotes loose coupling. – Enables flexibility. – Provides a common language between unrelated components. – Multiple inheritance of types.
- Defining interfaces: The `interface` keyword is used. E.g. `interface MyInterface { //methods }`
- Implementing interfaces: The `implements` keyword is used by classes that implement an interface. E.g. `class MyClass implements MyInterface { //methods }`

Here is the content in markdown format without any emojis or external links:

Exception Handling in Core Java

1. Exceptions: Exceptions are abnormal conditions that arise during the execution of a program. These exceptions disrupt the normal flow of the program.
2. Types of Exceptions: There are two types of exceptions:

- Checked Exceptions: Checked exceptions are checked at compile-time. These exceptions must be handled by the programmer. For example, IOException, SQLException, etc.
- Unchecked Exceptions: Unchecked exceptions are not checked at compile-time. These exceptions are usually runtime errors. For example, ArithmeticException, NullPointerException, etc.

3. Handling Exceptions: Exceptions can be handled in Java using try, catch and finally blocks.

- try block: The code that can potentially throw an exception is written in the try block.
- catch block: The catch block handles the exception. It must be preceded by a try block.
- finally block: The finally block contains code that will always execute after the try/catch block. It is used to release resources.

4. Throwing Exceptions: Exceptions can be thrown manually in Java using the throw keyword. The throw statement requires an exception object. For example, throwing a NullPointerException: throw new NullPointerException();
5. Creating Custom Exceptions: You can create your own custom exceptions by extending the Exception class or other exception classes. This enables you to handle specific exceptions in a customized way.

The content is written in points and in a formal tone without any emojis or external links as requested. Please let me know if you would like me to modify or expand the answer.

Here is the content in markdown format as per your instructions:

Multithread programming in Core Java.

1. Threads - A thread is a lightweight process that executes within a program and is used to multitask. Java supports multithreading that allows a program to perform multiple tasks simultaneously.
2. Creating Threads - There are two ways to create a thread in Java:

- Extending the Thread class: Create a class that extends Thread class and override the run() method.

- Implementing the Runnable interface: Create a class that implements Runnable interface and override the run() method. Then pass an instance of the class to the Thread constructor to create and start the thread.

3. Starting Threads - Once a thread is created, it will not start executing automatically. The start() method of Thread class is used to start the execution of a thread.
4. Thread State - A thread always exists in one of the following states:

- New: When a thread is instantiated, it is in new state.
- Runnable: A thread is in runnable state after start() is called but the thread scheduler has not selected it yet for running.
- Running: The thread scheduler has selected the thread and it is currently running.
- Blocked: A thread is blocked waiting for a lock, I/O, etc.
- Waiting: A thread is waiting for some condition to occur.
- Terminated: A thread has completed its task.

[No emojis or external links have been included as per the instructions.]

Here is the formal content on I/O in Core Java in markdown format without any emojis or external links:

I/O in Core Java

1. Java I/O (Input and Output) is used to process the input and produce the output.
2. There are two types of I/O:
    - Byte Stream I/O: Used to perform input and output of bytes. eg. FileInputStream and FileOutputStream.
    - Character Stream I/O: Used to perform input and output of characters. eg. FileReader and FileWriter.
3. Java uses streams to handle I/O. A stream is a sequence of data and the Java I/O streams are byte streams or character streams.
4. Some important classes for file I/O:
    - File - Used to create, delete, rename files/folders and get file attributes.
    - FileInputStream - Used to read bytes from a file.
    - FileOutputStream - Used to write bytes to a file.
    - FileReader - Used to read characters from a file.
    - FileWriter - Used to write characters to a file.
5. The File class is used to get file attributes such as name, path, size etc. but it cannot be used to read/write file contents. The FileInputStream/FileOutputStream and FileReader/FileWriter classes are used to read/write file contents.
6. While reading/writing files, it is important to close the stream after use using the close() method to release resources. Also, files should be flushed if only some buffered data is to be written using the flush() method.

Here is the content in Markdown format without any emojis or external links, written in a formal tone with points:

Java Applet in Core Java

1. Java applets are small Java programs that are embedded in HTML pages. They are executed within a Java-enabled browser.
2. Applets make web pages dynamic and interactive. They can include graphics, animation, and audio clips.
3. The tag is used to embed an applet into an HTML page. It has attributes like code, width, height, and alt.
4. The code attribute specifies the name of the applet's main class file. The width and height specify the applet's display area size.
5. Applets have some limitations:
    - They must be signed before running on a browser due to security reasons.
    - The user must have Java plug-in installed to run the applet.
    - Applets cannot access local system resources due to security restrictions.
    - Applets cannot interact with the user unless the page has a forms interface or applet methods are called from JavaScript.
6. Java applets are less popular now due to the reasons above. Java Web Start and JavaFX are modern alternatives to deploy rich Internet applications.

Does this fulfill your requirements? Let me know if you would like me to modify or expand the content in any way.

Here is the formal content on the topic "String handling in Core Java" in markdown format:

String handling in Core Java

1. Creating Strings:

- String literals: "Hello World"
- String object: new String("Hello World")

2. String Length:

- .length() method: int length = "Hello World".length(); //Outputs 11

3. Extracting substrings:

- .substring(beginIndex): str.substring(2); //Outputs "llo World"
- .substring(beginIndex, endIndex): str.substring(2, 5); //Outputs "llo"

4. Comparing Strings:

- .equals(): str.equals("Hello World"); //Outputs true

- .equalsIgnoreCase(): str.equalsIgnoreCase("hello world"); //Outputs true
- , >=, <, <=: Used for comparing strings lexicographically

5. Searching Strings:

- .indexOf(ch): str.indexOf('l'); //Outputs 2 (Index of first 'l')
- .lastIndexOf(ch): str.lastIndexOf('l'); //Outputs 3 (Index of last 'l')
- .contains(str): str.contains("World"); //Outputs true

6. Modifying Strings:

- .replace(oldChar, newChar): str.replace('l', 'x'); //Outputs "Hxello World"
- .toUpperCase(): str.toUpperCase(); //Outputs "HELLO WORLD"
- .toLowerCase(): str.toLowerCase(); //Outputs "hello world"

7. Splitting and Joining Strings:

- .split(regex): str.split(" "); //Outputs ["Hello", "World"]
- .join(array): String.join("-", {"Hello", "World"}); //Outputs "Hello-World"

Here is the content in markdown format with formal tone and without external links or emojis:

Event handling in Core Java.

1. Events: Events are notifications sent by objects to notify the occurrence of an action or an incident. In Java, events are handled through listeners.
2. Listeners: Listeners are interfaces that contain methods to handle events. They listen to the events and handle them accordingly when they occur. For example, ActionListener listens to action events.
3. Event Sources: These are objects that generate events. For example, a button is an event source that generates action events.
4. Registration: Listeners have to be registered with event sources in order to listen to and handle the events. This is done using methods like addActionListener().
5. Event handling: Once a listener is registered, it handles the events by implementing methods in the listener interface. For example, the actionPerformed() method in ActionListener is implemented to handle action events.
6. Unregistration: If required, listeners can be unregistered from event sources using methods like removeActionListener(). This is done to avoid memory leaks and extraneous events.

The steps for event handling are: 1. Define a listener interface 2. Write an event handling method in the listener interface 3. Define an adapter class that implements the listener interface 4. Register the adapter class as a listener with the event source 5. Unregister the listener if required

Event handling allows loose coupling between objects and enables event-driven programming. It makes applications more responsive and easier to maintain.

Here is the formal content in Markdown format without any emojis or external links as per your instructions:

Introduction to AWT in Core Java

1. AWT stands for Abstract Window Toolkit. It is a API to develop GUI or Graphical User Interface in Java.
2. AWT contains a rich set of GUI components like buttons, labels, textfields, containers, etc. which can be easily used in Java programs.
3. The java.awt package contains all the AWT classes and interfaces. Some key classes are:

- Window: Represents a top-level window.
- Frame: A Window that can have a title and window-management buttons.
- Panel: A generic lightweight container.
- Label: Displays a short string or an image.
- Button: A push button widget.
- TextField: Allows single line text input.

4. AWT components are rendered by the local system and hence follows the Look and Feel of the local system. The AWT widgets are heavyweight, i.e. they have native system-level representation.
5. The class hierarchy of AWT packages is complex with many interfaces and abstract classes. However, the components can be easily used by instantiating respective classes and adding them to the containers using the add() method.

The above content summarizes the key points about AWT in Core Java in a formal tone with points and without any emojis or external links as per your instructions. Please let me know if you would like me to modify or expand the content.

Here is the formal content on the topic “AWT controls” in markdown format:

AWT controls

1. Label: Used to display text/label. Does not accept any user input.
2. Button: Used to accept user commands via click. Has options like push button, radio button, check box, etc.
3. TextField: Used to accept single line text input from user.
4. TextArea: Used to accept multi-line text input from user.
5. List: Used to provide list of options to choose from. Has options like scroll list, select list, etc.
6. Checkbox: Used to accept boolean input from user. Status can be checked or unchecked.
7. Choice: Similar to list but can only choose one option.

8. Canvas: Used to draw shapes/graphics. Needs additional graphics context to draw on the canvas.
9. Scrollbar: Used for scrolling on components like text area.
10. Panel: Used as a container to group related controls.

The AWT controls provide basic UI elements to create graphical user interfaces and accept user input. They are platform-independent as they use native system peers to draw the components. They are heavyweight components and are not as flexible as Swing controls.

How's this? I have removed any friendly language or emojis and written the content in a formal tone with points and markdown format as instructed. Let me know if you would like me to modify or expand the content.

Here is the content in formal tone without emojis or external links, written in Markdown format with points:

Layout managers in AWT

1. FlowLayout: Components are arranged in a row or column, depending on the layout's orientation. spacing between components can be specified. This is default layout manager for Frame and Panel.
2. BorderLayout: Divides the container into five areas - NORTH, SOUTH, EAST, WEST and CENTER. Each area can contain only one component. Useful for simple layouts.
3. GridLayout: Arranges components in a grid of rows and columns. Each cell can contain only one component. Grid size can be specified in the constructor. Useful for creating forms.
4. CardLayout: Displays only one component at a time. Multiple components can be added and the visible component can be changed dynamically using first, next, previous, last methods. Useful for wizards.
5. GridBagLayout: Most flexible layout manager allowing components to span across multiple cells and have different sizes. Lays out components based on constraints. Complex to use but powerful. Useful for complex user interfaces.

The above points cover the key features and use cases of the common layout managers in AWT to arrange components in a container. Please let me know if you would like me to elaborate on any of the points or add more details.

Here is the content in markdown format without any emojis or external links, written in a formal tone with points:

Unit 2 - Web Page Designing

1. HTML: HTML stands for Hypertext Markup Language. It is the main markup language used to create web pages. HTML uses tags like

,

,

,

,

,

, etc. to structure a web page and its content.

2. CSS: Cascading Style Sheets (CSS) is used to style the HTML elements and define the layout of a web page. It separates the presentation of the content from the content itself. This makes web pages more adaptive and easier to maintain. CSS uses selectors and properties to style HTML elements.
3. JavaScript: JavaScript is a lightweight, interpreted programming or scripting language that is typically embedded into web pages. It allows web pages to be interactive and more dynamic. It can be used to create animations, rollovers, pop-up menus, calculators, and more. JavaScript code is run by the browser to react to certain events and modify the content and layout of the web page.
4. Web Page Design Process: The web page design process involves the following key steps:
5. Determine the goals and purpose of the web page.
6. Choose a design layout and overall structure for the web page.
7. Select graphics, colors, and typography for the web page.
8. Write the necessary HTML, CSS, and JavaScript code.
9. Test the web page in different browsers and screen sizes and make necessary adjustments.
10. Launch the web page.
11. Important Considerations: Some important things to consider when designing a web page are:

- Simplicity and minimalism
- Loading speed
- Responsive design
- Accessibility
- Device compatibility
- Security
- Search engine optimization

Here is the formal content on the topic “HTML in Web Page Designing” in markdown format without any emojis or external links:

HTML in Web Page Designing

1. HTML stands for Hypertext Markup Language. It is the main markup language used to create web pages.
2. HTML uses tags like
   ,
   ,
   ,
   ,
   ,
   , etc. to structure a web page and its content.
3. The
   tag specifies that this is an HTML document and is placed at the start and end of the document.
4. The
   tag contains metadata about the HTML document that is not displayed on the page. The
   tag inside the
   tag specifies the title of the web page that is displayed on the tab and in search engine results.
5. The
   tag contains the visible page content that is displayed on the web page. Headings are specified using
   to
   tags. Paragraphs are specified using
   tags.
6. HTML tags are used to add images, links, tables, forms, and other elements to the web page.
7. HTML is easy to learn and is used to create static web pages. However, for dynamic and interactive web pages, languages like JavaScript and PHP are required in addition to HTML.
8. HTML follows a hierarchical structure where tags are nested inside other tags to specify structure and semantics. Proper nesting of tags is important to create a valid HTML document.

The content is written in a formal tone with points and without any emojis or external links as per your instructions. Please let me know if you would like me to modify or expand the content in any way.

Here is the formal content in markdown format without any emojis or external links:

List in Web Page Designing

1. Ordered List: Lists where each item has a numeric sequence are called ordered lists. They are created using `<ol>` tags. Each list item starts with `<li>`.

2. Unordered List: Lists where each item is marked with a bullet point are called unordered lists. They are created using `<ul>` tags. Each list item starts with `<li>`.

3. Definition List: Lists that contain terms and their definitions are called definition lists. They are created using `<dl>`, `<dt>`, and `<dd>` tags. The `<dt>` tag defines the term and `<dd>` tag provides the definition.

4. Nested Lists: Lists can be nested within other lists to create a multi-level structure. This is done by nesting the `<li>` tags of one list within the `<li>` tags of another list. This can be done for both ordered and unordered lists.

The above points cover the key types of lists used in web page designing along with how to code them using HTML tags. Lists provide an efficient way to present a set of items in an organized manner. Their use enhances the readability and accessibility of web pages.

Here is the content in formal tone without emojis or external links in markdown format:

Table in Web Page Designing

1. Table is used to display data in rows and columns. It makes the data more organized and easy to read.
2. The HTML tag for table is `<table>`. It contains `<tr>` for table rows and `<td>` for table data/cells.
3. The border attribute is used to specify the width of the border around the table. The width and height attributes are used to specify the width and height of the table.
4. The `<caption>` tag defines a table caption. It is used to give a title to the table. It is placed immediately after the `<table>` tag.
5. The `<thead>` tag is used to define the heading of the table. It contains the heading rows. The `<tbody>` tag defines the body of the table containing the actual data rows.
6. The `<colgroup>` tag is used to group one or more columns in a table. We can apply styles to the grouped columns using this tag.
7. The `<col />` tag is used to define common attributes for a column. We can set width, style, etc. for the column. It is placed inside the `<colgroup>` tag.
8. The table cells can be spanned across multiple columns and rows using the `colspan` and `rowspan` attributes respectively.
9. Styling a table includes setting colors, backgrounds, widths, height, text alignment, borders, etc. for the table, rows, columns, and cells.

Here is the formal content on the topic “Images in Web Page Designing” in markdown format without any emojis or external links:

Images in Web Page Designing

1. Images add visual appeal to web pages and enhance the user experience. They make the content more engaging, easier to understand and interesting to read.
2. Appropriate image selection is important. Choose images that are relevant to the content and user interests. They should be of high quality, optimized size and load fast. Blurry, unrelated or excessive images negatively impact the user experience.
3. Image placement plays a key role in design and UX. Align images properly with the text. Do not place images and text too close or far apart. Leave sufficient white space around images for a clean look.
4. Use images for branding, creating sections, highlighting points or adding backgrounds. They make a web page more attractive but should complement the text and not dominate it.
5. Provide alternative text for images for accessibility. This helps search engines and screen readers to understand the images and content better. The alt text should describe the image concisely.
6. Compress and optimize images to reduce page load time. Choose the right image format like JPEG, PNG or GIF and optimal dimensions and resolution based on how the image will be used.
7. Ensure images have relevant names, titles and captions to appear properly in image searches and social media sharing. Descriptive titles and captions also add more context and keywords for SEO.

Here is the content in Markdown format without any emojis or external links and in formal tone:

Frames in Web Page Designing

1. Frames are used to divide a web page into multiple sections where each section can load separate HTML document.
2. The main advantage of using frames is that it reduces bandwidth usage because the common parts of web pages can be loaded just once in one frame and other frames can load only the changing parts.
3. Frames provide flexibility to design web pages and can isolate parts of the web page, like navigation bars or advertisements, making it easy to update.
4. However, frames also have some disadvantages like creating accessibility issues, affecting SEO, and causing problem with bookmarks and navigation.
5. To create frames,
   and tags are used. The
   tag specifies how the browser window should be divided into rows and columns and contains tags. Each tag specifies which HTML document should be loaded into that frame.
6. While using frames, it is important to specify target attribute in links so

that the linked document opens in the correct frame. Also, important content should be kept outside frames for accessibility.

The content summarizes key points about Frames in Web Page Designing in a formal tone with bullet points and without any emojis or external links. Please let me know if you would like me to modify or expand the content.

Here is the content in markdown format with formal tone and without emojis:

Forms in Web Page Designing

1. Introduction to Forms: Forms allow users to enter data. The data entered by users is sent to a server for processing and storage. Forms are a key part of user interactions on the web.
2. Elements of a Form: The key elements of a form are:

- Form tags: `<form>` and `</form>`
- Input elements: `<input>` like text fields, checkboxes, radio buttons, submit buttons, etc. to collect user input
- Labels: `<label>` to identify the purpose of input elements
- Buttons: `<button>` to submit the form or perform certain actions

3. Different Types of Form Input Elements: There are many input types like:

- Text fields: To enter text data like name, email, password, etc.
- Checkboxes: To select one or more options from a list
- Radio buttons: To select only one option from a list
- Text areas: For entering long text inputs
- Date pickers: To select dates
- Color pickers: To select colors
- Submit buttons: To submit the form
- Reset buttons: To reset the form fields to initial values

4. Form Structure: The structure of a basic form includes:

- `<form>` tags to indicate the start and end of the form
- Input elements to collect user data
- Labels to identify inputs
- Buttons to submit or reset the form The inputs and buttons can be arranged in suitable structure using div tags or other layout elements.

5. Processing Form Data: The form data is processed on the server using a scripting language like PHP, ASP, etc. The form data is sent to the server as name-value pairs and can be accessed on the server to store in databases or perform certain actions.

Here is the content in formal tone with markdown format without emojis and external links:

CSS in Web Page Designing

1. CSS stands for Cascading Style Sheets. It is used to style the web pages and control the layout of web pages.
2. With the help of CSS, we can define styles for multiple HTML elements at once in a single CSS file. This makes it easy to change the look and feel of the whole web page.
3. Some of the common properties of CSS are:

- Font-family: To set the font type like arial, calibri, etc.
- Color: To set the text color.
- Background-color: To set the background color of the web page or specific elements.
- Font-size: To set the font size.
- Text-align: To align the text left, right or center.
- Padding: To add space around the content.
- Margin: To add space outside the border.
- Border: To add a border around the elements.

4. There are 3 ways to add CSS to HTML:

- Inline Styles: Using the style attribute in HTML elements.
- Internal Stylesheet: Using
  tag in the HTML page.
- External Stylesheet: Using a .css file and linking it in the HTML page.

5. CSS has various advantages like easy maintenance, faster loading web pages, device independence, accessibility, etc.

This content is written in formal tone with points and no emojis or external links as per the given instructions. The content is written inside the specified header in markdown format. Please let me know if you would like me to modify or expand the content.

Here is the formal content on the topic "Document type definition in Web Page Designing" written in Markdown format without any emojis or external links:

Document type definition in Web Page Designing

1. A document type definition (DTD) is a set of markup declarations that define a document type for an SGML-family markup language (GML, SGML, XML, HTML). A DTD defines the valid building blocks of an XML document. It defines the document structure with a list of valid elements and attributes.

2. A DTD can be declared inline inside an XML document, or as an external reference. The external DTD is preferred as it can be used by multiple XML documents. The DTD is defined inside a DOCTYPE declaration which is included in the XML declaration at the start of the document.

3. The benefits of a DTD are:

- It defines the structure of an XML document and validates that each document follows the structure.
- It acts as a contract between the document and its processing application.
- It enables the processing application to check for errors and ensure conformance.
- It serves as documentation for the structure and syntax of the elements and attributes of an XML document.

4. A DTD can use element declarations to declare elements, attribute declarations to declare attributes, entity declarations to declare entities, and notation declarations to declare notations. It uses a specific syntax with declarations of elements, attributes, entities, and notations within brackets. A basic example of an external DTD is:

<!DOCTYPE note SYSTEM “note.dtd”>

Tove Jani Reminder

Don’t forget me this weekend!

Here is the content in markdown format without any external links or emojis:

XML in Web Page Designing

1. XML stands for Extensible Markup Language. It is a markup language much like HTML.
2. XML was designed to carry data, not to display data. XML tags are not predefined like in HTML. You can define your own tags.
3. XML documents have a tree structure like HTML documents. They contain elements (tags) and attributes.
4. XML uses a DTD (Document Type Definition) to define the structure of the XML document. It defines which elements and attributes are allowed and which are not.
5. XML has many benefits:

- It is platform independent and programming language independent.
- It is easy to read and understand.
- It has a logical structure.
- It can handle complex data structures.
- It can transform data to HTML using stylesheets (XSLT).
- It is extensible. You can define your own tags and elements.

6. XML is widely used in web page designing to create RSS feeds, SVG images, and Ajax applications. Many web services return data in XML format because of the above-mentioned benefits of XML.

The content is written in points and in a formal tone without any emojis or external links as per your instructions. Please let me know if you would like me

to modify or expand the content further.

Here is the content in formal tone without emojis or external links, written in Markdown format with points:

DTD in Web Page Designing

1. DTD stands for Document Type Definition. It is a set of markup declarations that define a document type for an SGML-family markup language (SGML, XML, HTML).
2. A DTD defines the document structure with a list of valid elements and attributes. It specifies the rules for the structure and the elements and attributes that can be used in an XML or HTML document.
3. Having a DTD has a few benefits:

- It verifies that the document is structurally correct
- It provides rules/guidelines for formatting the document
- It allows validation - checking if the document follows the rules of the DTD

4. There are two ways to add a DTD to an HTML document:

- Internal DTD - defined inside the document in the head section
- External DTD - defined in a separate file and referenced from the document

5. A sample internal DTD would look like: <!DOCTYPE html [<!ELEMENT html (head,body)> <!ELEMENT head (title)> <!ELEMENT title (#PCDATA)> <!ELEMENT body (p)> <!ELEMENT p (#PCDATA)>]>
6. A sample external DTD reference in an HTML document would be: <!DOCTYPE html SYSTEM “sample.dtd”>

Does this content meet the required guidelines? Let me know if you would like me to modify or expand the content in any way.

Here is the content in markdown format without any emojis or external links as per your instructions:

XML schemes in Web Page Designing

1. XML stands for Extensible Markup Language. It is a markup language designed to store and transport data.
2. XML uses tags similar to HTML to define data elements. However, XML tags are not predefined like in HTML. You can define your own XML tags to store data.
3. Some key features of XML are:

- It is a self describing language. The XML document contains metadata that describes the structure of the data.

- It is extensible. New elements and attributes can be defined easily in XML.
- It is platform independent and programming language independent.
- It can be processed easily by machines.

4. Some applications of XML are:

- RSS feeds
- SVG images
- Office documents
- Configuration files
- Databases
- Web services

The content is written in point form, using a formal tone and without any emojis or external links as per your instructions. Please let me know if you would like me to modify or expand the content in any way.

Here is the content in Markdown format without emojis or external links:

Object Models in Web Page Designing

1. Document Object Model (DOM):

- The DOM is a programming interface for HTML and XML documents.
- It represents the page so that programs can change the document structure, style, and content.
- The DOM is constructed as a tree of nodes: – The root node is the HTML element. – Every element, attribute, and piece of text in the HTML is a node in the tree. – Each node can have child nodes. – Programmers can change every aspect of a page by manipulating the DOM tree.

2. Cascading Style Sheets Object Model (CSSOM):

- The CSSOM represents the structure of a style sheet.
- It converts the CSS code into a format that can be accessed and manipulated by programs.
- Programmers can use the CSSOM to dynamically read and change the styles in a web page.

3. Graphics Object Model (Graphics):

- The Graphics object model represents the shapes and images in a web page.
- It allows programmers to dynamically create and change graphics in a page.
- Common uses include creating animations and interactive graphics.

The content is written in formal tone as a study material without any emojis or external links and in Markdown format with points. Please let me know if you would like me to modify or expand the content.

Here is the formal content on the given topic in markdown format:

Presenting and Using XML in Web Page Designing

1. XML stands for Extensible Markup Language. It is a markup language designed to store and transport data.
2. XML tags are not predefined like HTML tags. Users can define their own tags and attributes as per their requirements. This makes XML flexible and extensible.
3. XML documents have a tree structure with a root element. All other elements are children of the root element.
4. XML uses tags similar to HTML to define elements and attributes. However, XML tags are case-sensitive and must be properly nested.
5. XML can be used to store data for websites. The data can be stored in XML files and extracted using XML parsers to be displayed on web pages.
6. XML data can be easily transformed into HTML using Extensible Stylesheet Language Transformations (XSLT). This makes it suitable for designing web pages.
7. The semantic nature of XML makes the data more understandable by machines. This enables easier processing of XML data using programming languages.
8. Some applications of XML on the web include RSS feeds, SVG, and AJAX. It is a preferred format for data interchange on the web due to its simplicity and flexibility.

The content is written in formal tone with points and without any emojis or external links as instructed. The markdown formatting is used and the content is written inside the specified header. Please let me know if you would like me to modify or add anything to the content.

Here is the formal content in Markdown format without any emojis or external links:

Using XML Processors in Web Page Designing

1. XML Parsing: XML documents are parsed using XML processors called parsers. Parsers read the XML document and convert it into an XML parse tree or DOM (Document Object Model) which can be easily processed by programming languages.
2. DOM vs SAX: There are two types of XML parsers - DOM parsers that load the entire XML document into memory and create a DOM, and SAX parsers that sequentially read the XML document and trigger events for each part of the document. DOM parsers are easier to use but have more memory overhead while SAX parsers have lower memory overhead but are more complex to use.
3. Processing the DOM: Once the DOM is created, it can be processed using programming languages to extract the required information, modify ele-

ments or values, add or delete elements, etc. Various XML DOM methods can be used to traverse the tree and achieve the required functionality.
4. Transforming XML data: XML data can be transformed into other formats like HTML or other XML formats using XSLT stylesheets. XSLT processors take the XML document and the XSLT stylesheet as input and generate the required output by applying the transformations specified in the stylesheet.
5. Validation: XML documents can be validated against a DTD or XML Schema to ensure that the document structure matches the grammar. This is done by XML validators which take the XML document and schema as input and check if the document is a valid instance of the schema.

The above points cover the key steps involved in processing XML documents for web page designing. Please let me know if you would like me to elaborate on any of the points or modify the content in any way.

Here is the formal content on DOM and SAX in Web Page Designing:

DOM and SAX in Web Page Designing

1. DOM or Document Object Model:

- DOM is a programming interface for HTML and XML documents.
- It represents the page so that it can be accessed and manipulated by programming languages.
- The DOM views the HTML document as a tree of nodes.
- It creates a full object model of the HTML document which can be used to access and modify the structure and content of the document.
- DOM is easier to use but requires loading the entire document into memory, so it is memory intensive and slower.

2. SAX or Simple API for XML:

- SAX is an event-based sequential access parser API for XML.
- It does not load the entire document into memory at once. Instead, it reports parsing events to the application.
- The application can handle the events to read/write portions of the document as it is needed.
- So, it does not require much memory but the application has to maintain its own state since it only gets fragments of the document.
- SAX is more efficient and faster but more complex to use compared to DOM.

The choice between DOM and SAX depends on the application requirements like speed, memory usage, simplicity, etc. Both have their pros and cons and can be used for parsing XML and HTML documents to access and manipulate their content.

Here is the formal content on the topic “Dynamic HTML in Web Page Designing”

in markdown format without any emojis or external links:

Dynamic HTML in Web Page Designing

1. Dynamic HTML or DHTML is the combination of HTML, CSS, and JavaScript to create interactive and dynamic web pages.
2. It allows web pages to be created dynamically on the client-side by JavaScript which can change the page content and layout without loading the page again from the server.
3. Some features of DHTML are:

- Changing color and style of the elements
- Showing/hiding elements
- Resizing and moving elements
- Creating animations
- Validating web forms
- Creating sticky headers and image sliders

4. DHTML allows web pages to become more responsive, engaging, and customized without the need to refresh the web page.
5. However, DHTML also has some limitations like variation in behavior across different browsers and inconsistent performance. It also makes web pages dependent on client-side languages like JavaScript which can be disabled.
6. DHTML is being used to create single page applications and sophisticated web interfaces. It has greatly enhanced the user experience of web pages and made web development more interesting and creative.

Does this look okay? Let me know if you would like me to modify or add anything.

Here is the formal content in markdown format without any emojis or external links under the header ## Unit 3 - Scripting:

Unit 3 - Scripting

1. Scripting is a way to automate tasks using a scripting language. Some benefits of scripting are:

- Increased productivity and efficiency. Repeatable and routine tasks can be automated.
- Fewer errors. Manual steps are replaced by scripts which can run accurately every time.
- Scalability. Scripts can be run multiple times to handle large volumes of tasks.

2. Popular scripting languages include:

- Bash - Used for scripting on Linux and macOS.

- Python - A general purpose and easy to learn language used for both scripting and programming.
- JavaScript - Originally used for web scripting but can also be used as a general scripting language.
- PHP - Used for scripting web pages on a server.
- PowerShell - A scripting language for automating and managing Windows and Windows Server systems.

3. Some useful concepts and tools in scripting include:

- Variables to store data
- Control flows like if-else conditions and loops to control the logic
- Functions to modularize code and reuse it
- Automation of user inputs and system commands
- Reading and writing files
- Handling errors and exceptions
- Logging and debugging

4. Scripting is used extensively in IT for automation. Some examples are:

- Automation of backups and deployment processes
- Automation of software testing
- Automation of monitoring and alerts
- Automation of reports generation
- Automation of DevOps tasks

Does this content meet your requirements? Let me know if you would like me to modify or add anything.

Here is the formal content on the topic "Java script in Scripting" written in points and markdown format without any emojis or external links:

Java script in Scripting

1. Java Script is a lightweight, interpreted programming or scripting language that is typically embedded into web pages.
2. It allows web pages to be interactive and more dynamic.
3. Java Script code is executed by a browser's JavaScript engine.
4. It can execute not only after a web page has loaded but also as soon as specific events occur, for example when a user clicks on an HTML element or when a web page finished loading.
5. Java Script is different from Java. Java is a compiled programming language whereas Java Script is an interpreted scripting language.
6. Java Script is widely used to create interactive effects within web browsers and to enhance the user experience.
7. Some of the common things done using Java Script are:

- Validating web forms before submitting them.
- Displaying popup messages.

- Redirecting web pages.
- Detecting the visitor's browser and its capabilities.
- Maintaining the state of web applications.
- Creating cookies.

I have written the content in points in a formal tone without any emojis or external links as instructed. Please let me know if any changes are required.

Here is the content in formal tone without emojis or external links, written in markdown format with headings:

Introduction to JavaScript

1. JavaScript is a lightweight, interpreted programming or scripting language that is typically embedded into web pages.
2. JavaScript is designed to run on a browser and make web pages more interactive.
3. JavaScript code can react to user events such clicking buttons, filling out forms, and other actions.
4. JavaScript can access and manipulate content and structure of a web page.
5. JavaScript borrows some syntax from C and Java languages, but it is otherwise unrelated to Java.
6. JavaScript runs on a browser, not on a server. It is lightweight and interpreted, not compiled.
7. JavaScript can be embedded directly into HTML pages or included in external .js files.
8. JavaScript is an easy to learn and flexible language widely used to create interactive effects within web pages and applications.

I have written the content in points and in a formal tone as per your instructions. Please let me know if you would like me to modify or expand the content in any way.

Here is the content in Markdown format with points, without emojis or external links, in a formal tone:

Documents in JavaScript

1. HTML documents - JavaScript code is embedded into HTML documents and interacts with the HTML elements and attributes. The JS code makes the web page more interactive and dynamic.
2. JavaScript files - The JS code can also be written in separate .js files and linked to the HTML documents. The .js file contains the logic and functionality while the HTML file contains the structure.
3. Inline JavaScript - The JS code can also be written inline directly within the HTML tags or attributes. However, this is not a best practice as it mixes structure and logic and makes the code harder to read and maintain.

4. Client-side JavaScript - The JavaScript code is executed on the client-side, i.e. in the user's browser. It enhances the user experience and interactivity of web pages and web applications.
5. Server-side JavaScript - JavaScript can also be executed on the server-side through the use of runtime environments like Node.js. In this case, JavaScript can be used to build server-side applications and dynamic page generation logic.

The above points cover the key types and uses of JavaScript documents for learning and reference. Please let me know if you would like me to elaborate on any of the points or modify the content in any way.

Here is the content in markdown format with formal tone and without any emojis or external links:

Forms in JavaScript

1. Introduction to Forms

- Forms are a collection of form elements/controls that allow users to enter data.
- Form elements include input fields, radio buttons, checkboxes, submit buttons, etc.
- The data from forms is sent to a server for processing using HTTP methods GET or POST.

2. Accessing Form Elements

- Form elements can be accessed using their id, name, class, tag name, etc.
- For example: document.getElementById('id'), document.getElementsByName('name'), document.getElementsByClassName('class'), etc.
- The elements can then be read or modified using properties and methods.

3. Submitting a Form

- The form is submitted using the submit() method on the form.
- For example: formObject.submit()
- The submitted data can be accessed on the server side to be processed.
- Preventing default submission action: The default submission behavior can be prevented using e.preventDefault() on the submit event. This is useful when you want to perform client side validation or processing before sending the data to the server.

4. Form Validation

- Form validation is essential to ensure correct user input.
- It can be done on both client side (using JavaScript) and server side (using a server side programming language like PHP, Java, etc.).
- Client side validation is faster but can be bypassed, server side validation is more secure but results in latency.

- Common validations include checking for required fields, data type, length, format, etc.

5. Conclusion

- Forms are an important way to collect user input data.
- JavaScript can be used to access form elements, modify them and validate user input.
- The data can then be sent to the server for subsequent processing using HTTP methods.

Here is the content in the required formal style: #### Statements in JavaScript 1. JavaScript statements are executed sequentially, one after another, unless control flow statements change the flow of execution. 2. A statement can be a: - Simple statement: Expression or control flow statement - Block statement: Set of statements enclosed in braces {} 3. Expression statements: An expression that is evaluated and produces a value. For example: a=7; x=y*z; 4. Control flow statements: Include conditionals, loops and exceptions that control the flow of execution. - Conditional statements: if/else and switch evaluate conditions and execute code based on the results. - Loop statements: for, while and do…while repeatedly execute a block of code. - Exception handling statements: try, catch and throw handle errors and exceptions. 5. JavaScript statements must be terminated by a semicolon (;). Semi-colons are optional in some cases but including them improves readability and avoids errors.

The content is written in markdown format with bullet points, no emojis or external links are included and a formal tone is maintained with no feelings expressed. The content is written inside header statements as specified. Please let me know if any other changes are required.

Here is the formal content on the topic “Functions in JavaScript” in markdown format:

Functions in JavaScript

1. A function is a block of code that performs a specific task.
2. Functions allow you to organize your code into logical blocks, make it more readable, reusable and efficient.
3. Function syntax: function name(parameter1, parameter2) { // code to be executed }
4. Parameters are the inputs for a function and are optional.
5. You can pass values to functions as arguments. If the number of arguments does not match the number of parameters, the function may produce unexpected results.
6. Functions can return a value using the return statement. The return statement exits the function and passes back the value to the calling function.

7. Example of a function that takes two numbers and returns their sum: function sum(num1, num2) { return num1 + num2; }
8. To call or execute the function, use its name followed by parentheses: sum(5, 10); // Returns 15
9. Functions can be reused and called multiple times. This makes your code more efficient and less repetitive.
10. JavaScript has many built-in functions such as parseInt(), alert(), etc. You can also create your own functions.

Functions allow you to break down complex problems into small steps and build reusable code. They make your JavaScript programs more modular, flexible and organized.

Here is the content in the requested formal style without emojis or external links: #### Objects in JavaScript 1. Objects are collections of related data and/or functionality (methods) that act as a single unit. 2. Objects are instantiated from classes/functions and they encapsulate data with code to work on that data. 3. An object can have properties (values) and methods (functions). Properties store values and methods are actions that can be performed on the object. 4. Objects are mutable (can be changed) and multivalued (can have multiple properties and methods). 5. Objects are instances of classes/functions. The class/function acts as a blueprint for the object. 6. Data encapsulation allows an object's properties/methods to be accessed only through defined public interfaces. This hides the internal implementation details of the object. 7. JavaScript uses function scope: All variables defined inside a function are only accessible within that function. However, objects created in functions can maintain state beyond the function. 8. The properties and methods of an object can be accessed using the dot or bracket notation. Dot notation is easier but bracket notation can be used to access properties/methods with spaces or special characters. 9. JS has multiple built-in objects such as String, Number, Math, Date etc. These objects have properties and methods to perform various tasks. We can also create custom objects. 10. Object oriented programming facilitates code reuse, modularity and logical structuring. However, JavaScript uses a prototype-based inheritance model instead of a classical inheritance model.

Here is the content in formal tone with points in Markdown format:

Introduction to AJAX in JavaScript

1. AJAX stands for Asynchronous JavaScript and XML.
2. It is the use of the XMLHttpRequest object to communicate with server/web pages in the background.
3. It can send and receive data in various formats, including JSON, XML, HTML, and text files.
4. The key feature is that it is asynchronous, meaning it does not block the user from interacting with the web page while the communication is happening.

5. The steps to implement AJAX are:

- Create an XMLHttpRequest object.
- Create a callback function to handle the server response.
- Open a connection to the server.
- Send the request.
- Listen for the response.

6. The benefits of AJAX are:

- Increased interactivity.
- Faster performance by updating only portions of a web page.
- Reduced need for page refreshes.
- Simulates a desktop app feel.

7. AJAX limitations and concerns are:

- Browser compatibility issues.
- Security issues if data is sensitive.
- Increased complexity of web pages.
- Reliance on JavaScript being enabled.

8. Common uses of AJAX are:

- Automatic updates of web pages.
- Webmail.
- Commenting systems.
- Search engine suggestions.
- Online maps.

Here is the formal content on the topic "Networking in Scripting" in markdown format:

Networking in Scripting

1. Socket Programming: Scripting languages like Python and Perl provide libraries to create socket connections between computers. Sockets can be used to create client-server applications for communication over networks. Scripts can be written to create TCP/IP and UDP socket connections to transfer data.
2. HTTP Requests: Scripts can be used to send HTTP requests to fetch data from web servers and APIs over the network. The received data can then be parsed and processed as required. This enables scripting languages to access network resources using the HTTP protocol.
3. SSH Connections: Scripts can be used to make SSH connections to remote servers and run commands or transfer files over the secure SSH protocol. This allows scripting languages to log into remote systems and control them over the network.
4. Scraping Websites: Scripts can be used to scrape data off websites by sending HTTP requests and parsing the HTML content received. This

enables the script to extract information from web pages over the network. The scraped data can then be processed as needed.
5. Sending Emails: Scripts can use SMTP libraries to send emails over the network to any email address. This allows the script to use the email network to notify users or send reports/alerts as required.

The above points cover some key ways in which scripting languages can be used to access networks and resources over the network. Scripts can leverage various network protocols and libraries to communicate over the network for automation and other tasks.

Here is the content in markdown format with formal tone and without emojis:

Internet Addressing in Networking

1. IP Address: An IP address is a numerical label assigned to each device connected to a network that uses the Internet Protocol for communication. It comprises of a network prefix and a host identifier. IP addresses are categorized into versions - IPv4 and IPv6. IPv4 uses 32-bit addresses while IPv6 uses 128-bit addresses.

2. Subnet Mask: A subnet mask is combined with an IP address to identify two parts - the extended network address and the host address. It helps in identifying the network and host portions of an IP address. The subnet mask contains consecutive 1s followed by consecutive 0s. The number of 1s denotes the significant part of the address used for identifying the network.

3. CIDR Notation: CIDR (Classless Inter-Domain Routing) notation is a compact representation of an IP address and its associated routing prefix. It replaces the traditional classful network address format. It conjoins the IP address and its subnet mask into a single CIDR prefix notation. For example, 192.168.1.0/24 refers to an IPv4 address block with a 24-bit prefix and 8-bit host identifier.

4. Private IP Addresses: Private IP addresses are used for local networks and are not routable on the public Internet. They help conserve the limited IPv4 address space. The three ranges of private IPv4 addresses are 10.0.0.0-10.255.255.255, 172.16.0.0-172.31.255.255 and 192.168.0.0-192.168.255.255.

5. Public IP Addresses: Public IP addresses are unique IP addresses allocated by regional internet registries to enterprises or internet service providers for use on the Internet and its public-facing servers and network devices. They are routable on the public Internet.

Here is the content in formal tone with points in Markdown format:

InetAddress in Networking

1. InetAddress represents an Internet Protocol (IP) address. It can be an IPv4 address or an IPv6 address.
2. InetAddress Class provides methods to get the IP address in numeric and string formats. It helps in converting the IP address from one format to another.
3. Methods in InetAddress:

- getAddress() - Returns the raw IP address in an integer array.
- getHostAddress() - Returns the IP address in a string format.
- getHostName() - Returns the hostname.
- isReachable(int timeout) - Checks if this address is reachable in the given timeout in milliseconds.
- equals(Object obj) - Compares this object against the specified object.

4. InetAddress can be used to determine the host name associated with an IP address, perform hostname resolution, and check whether a particular hostname is reachable.
5. Few ways to create an InetAddress object:

- InetAddress.getLocalHost() - Returns InetAddress representing local host.
- InetAddress.getByName(String host) - Returns InetAddress representing the host.
- new InetAddress(byte[] addr) - Creates an InetAddress from raw IPv4 or IPv6 address.

The points are written in formal tone with no emoji or external links. The content is written inside the specified header in Markdown format. Please let me know if you would like me to modify or add anything.

Here is the content in the required format:

Factory Methods in Networking

1. Factory methods are creational design patterns that solve the problem of creating objects without having to specify the exact class to create.
2. In networking, factory methods are commonly used to create connections to servers or clients. For example, a factory method can be used to create a connection to a database without specifying the exact database vendor (e.g. MySQL, PostgreSQL, etc.). The factory method chooses the appropriate connection based on the provided parameters.
3. Using factory methods for connections has several advantages:

- It decouples the code from specific implementations. The code uses the generic factory method interface and is not dependent on specific connections.
- It makes the code more maintainable by isolating connection creation in a single place. If a different connection type is needed in the future, only the factory method implementation needs to change.

- It can choose an optimized connection implementation based on run-time parameters (load, resource usage, etc.).

4. Some common examples of factory methods for networking are:

- Database connection factories - Creating connections to MySQL, PostgreSQL, etc. database servers.
- Socket factories - Creating connections using various transport protocols (TCP, UDP, etc.).
- Web service clients - Creating clients for various web service implementations (SOAP, REST, etc.).

The above content is written in a formal tone with points and without any emojis or external links as per the given requirements. The header for the topic is also included. Please let me know if you would like me to modify or expand the content in any way.

Here is the content written in formal tone without any emojis or external links in Markdown format:

Instance Methods in Networking

1. getHostName() - Returns the hostname of the server.
2. getPort() - Returns the port number of the server.
3. getLocalAddress() - Returns the local address of the server.
4. getInetAddress() - Returns the IP address of the server.
5. getLocalPort() - Returns the local port number of the server.
6. getRemoteSocketAddress() - Returns the remote socket address of the server.

The above are some of the commonly used instance methods available in the networking package to fetch various details about the server. These methods can be invoked on the server object to extract the required information in a formal programming setup.

How's this? I have written the content in points and included the relevant instance methods along with a brief description of each method. Let me know if you would like me to modify or add anything.

Here is the formal content on the topic "TCP/IP Client Sockets in Networking" without any emojis or external links:

TCP/IP Client Sockets in Networking

1. TCP/IP is the underlying protocol suite for network communications on the Internet. It provides reliable, ordered, and error-checked delivery of transmitted data.
2. A socket is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so that the OS can identify the application to which the data is sent.

3. A client socket is used by a client application to connect to a server application to exchange data. To create a client socket, the client application calls the socket() system call to obtain a socket descriptor.
4. The client uses the connect() system call to initiate a TCP connection to the server socket by specifying the server IP address and port number. If the connection is successful, the client socket is connected to the server socket and the two sockets can exchange data.
5. The send() and recv() system calls are used to send and receive data through the connected client socket. The client and server can send and receive data concurrently and in both directions.
6. When the communication is complete, the close() system call is used to release the socket resources and close the connection.

The above content summarizes the key steps involved in using TCP/IP client sockets for network communications without any personal touches or external references. Please let me know if you would like me to elaborate on any of the points or modify the content in any way.

Here is the content in formal tone with points and without emojis or external links:

URL in Networking

1. URL stands for Uniform Resource Locator. It is a string of characters that identifies the location of a resource on the Internet.

2. The basic structure of a URL consists of 5 components:

Scheme - Defines the protocol to be used to access the resource e.g. http, https Host - Domain name of the server hosting the resource e.g. www.example.com Port - Optional part specifying the port number to be used if other than default Path - Path to the resource on the server e.g. /path/to/resource Query - Optional string containing parameters and values for searching/fetching the resource

3. URLs allow clients to access resources stored on servers connected to the Internet. They provide a standardized way to specify the location of a resource and enable the routing of requests to reach the desired server that stores the resource.

4. Depending on what a URL refers to, it can be classified as:

Web URL - Refers to a web page Email URL - Refers to an email address FTP URL - Refers to a file on an FTP server

That's the content on the topic of URL in Networking written in formal tone with points and without emojis or external links. Let me know if you would like me to modify or expand the answer.

Here is the content in markdown format with formal tone and without any emojis or external links:

URL Connection in Networking

1. URL stands for Uniform Resource Locator. It is an address that specifies the location of a resource on the Internet.
2. A URL has four components:
   - Scheme - It specifies the protocol to be used to access the resource. For example, http, https, ftp, etc.
   - Host - It specifies the domain name of the server that hosts the resource.
   - Port - It specifies the port number to connect to the server. If not specified, default port numbers are used (80 for http and 443 for https).
   - Path - It specifies the location of the resource on the server.
3. To access a resource on the Internet, a connection is made to the server using the URL. The server then sends a response which can contain the requested resource (web page, image, video, etc.) or an error if the resource cannot be found.
4. URL connections enable clients (web browsers) to access and fetch resources from servers over the Internet using standard protocols. They form the basis of how the World Wide Web functions.

How's this? I have written the points in a formal tone without any emojis or external links as requested. Please let me know if you would like me to modify or add anything.

Here is the formal content written in Markdown format inside the requested header:

TCP/IP Server Sockets in Networking

- TCP/IP is the communication language or protocol of the Internet. It can also be used as a communications protocol in a private network.
- TCP provides reliable, ordered, and error-checked delivery of transmitted data.
- IP handles addressing and routing of data. It gets data to the correct destination network and host.
- A socket is one endpoint of a two-way communication link between two programs running on the network.
- A server socket listens for requests to come in from clients. When a request arrives, the server accepts it and performs some service, like retrieving a web page or sending an email, before sending a response back to the client.
- To create a server socket, you need to know the port number. Well-known ports are specific ports assigned to specific servers, for example, port 80 is used for web servers.

- Server sockets enable a server program to receive incoming requests and respond to them. They allow a server program to listen for incoming connections and accept them. Once a connection is accepted, the server can send and receive data through the socket.

The above points cover the key highlights of TCP/IP server sockets in networking in a formal tone without any feelings or friendliness as requested. The content is written in Markdown format with bullet points and no emojis or external links have been included. Please let me know if you would like me to elaborate on any of the points or modify the content in any way.

Here is the content in markdown format as per your instructions:

Datagram in Networking

1. A datagram is a self-contained, independent packet of data that includes source and destination address and contains the data being sent.
2. Datagrams are the units of data that are transmitted across the network in a packet switching network.
3. Datagrams are created whenever there is data to be sent. The datagrams are then transmitted across the network individually and can arrive out of order or not at all.
4. Datagrams are connectionless and do not establish a dedicated end-to-end connection between two hosts. Each datagram is treated as an independent unit of data without any relationship to previous or subsequent datagrams.
5. In User Datagram Protocol (UDP), datagrams are used where speed and efficiency are priorities and some loss of data is acceptable. UDP is suitable for applications such as video streaming that require fast delivery.
6. In Transmission Control Protocol (TCP), reliable in-order delivery of data is required so datagrams are sequenced and acknowledgements are used. TCP provides a reliable connection-oriented service suitable for applications such as file transfer and email.

The content is written in points and in a formal tone as per the given instructions. Emojis, external links and informal language have not been used. The content is aimed to be like study material to learn and read about datagrams in networking. Please let me know if you would like me to clarify or expand on any of the points.

Here is the formal content written in Markdown format without any emojis or external links under the header ## Unit 4 - Enterprise Java Bean:

Unit 4 - Enterprise Java Bean

1. Enterprise JavaBeans (EJB) is a development architecture for multi-tier enterprise applications.
2. EJB runs on top of an application server and provides services such as transactions, security and scalability.

3. The EJB architecture focuses on reusability of components and a separation between the business logic and the presentation layer.
4. The main advantage of EJB is that it reduces complexity by providing a managed environment for developing enterprise grade applications.
5. The different types of EJBs are:

- Session Beans: Used to encapsulate business logic. Two types: Stateless and Stateful
- Message Driven Beans: Used to provide asynchronous communications via messages
- Entity Beans: Used to map objects to a relational database. Deprecated in EJB 3.1

6. An EJB application typically consists of an EJB container and multiple EJB modules. The EJB container provides services to the EJB modules and manages their lifecycle.
7. EJB implements a client-server architecture. The application client uses the EJB components remotely via interfaces. The EJB components run inside the EJB container.
8. The different EJB interfaces are:

- Remote: Used for remote access. The business methods are accessed remotely via RMI.
- Local: Used for local access. The business methods are accessed locally via direct method calls.
- No-interface: Used since EJB 3.1 to develop POJOs as EJBs. Only local access is possible.

9. EJB supports dependency injection to obtain resource references and can be packaged as deployable units (EJB JAR or WAR file). It provides metadata via XML deployment descriptors or annotations.

Here is the content in markdown format with formal tone and without emojis:

Preparing a Class to be a JavaBeans

1. The class should be public.
2. The fields should be private with getter and setter methods (also known as accessor and mutator methods).
3. The fields should be typed and have meaningful names.
4. Provide a no-arg constructor.
5. The methods should be public.
6. The class should have a serializable interface.

To recap, to prepare a class to be a JavaBean, follow these steps:

1. Declare the class as public.
2. Declare the fields as private.
3. Provide getter and setter methods for the fields.

4. Use meaningful names and types for the fields.
5. Provide a no-arg constructor.
6. Declare the methods as public.
7. Implement the Serializable interface.

Following these steps will ensure the class can be reused and integrated in an application. The getter and setter methods provide encapsulation while allowing the fields to be accessed as needed. The serializable interface allows the object to be persisted. By being public and providing a no-arg constructor, the class can be instantiated as needed.

Here is the content in markdown format without emojis or external links, written in a formal tone with points:

Creating a JavaBeans

1. Define the class: Create a public class with a name that follows Java naming conventions (start with a capital letter, camelCase).
2. Add properties: Add private member variables (aka fields) to hold the state/data of the bean, with get and set methods (getter and setter methods) to access them. The get and set methods should also follow Java naming conventions.
3. Add a no-arg constructor: Create a public no-argument constructor so that the bean can be instantiated without supplying any arguments.
4. Make the class serializable: If you want the bean to be serializable, implement the Serializable interface. This allows the bean to be converted to and from a byte stream so that it can be stored or transferred over a network connection.
5. Build accessor methods: Create public get and set methods for accessing and mutating the member variables of the bean. This encapsulates the internal state of the bean and allows controlled access to its data.

The steps above describe how to create a basic JavaBeans class. Following standard naming conventions and encapsulation practices leads to classes that can be used in a plug-and-play fashion, allowing reuse and decoupling of components.

Here is the content in formal tone without emojis or external links, written in Markdown format with points:

JavaBeans Properties

1. JavaBeans are reusable software components for Java. They have a standard way of interacting with each other.
2. JavaBeans have properties that can be set and retrieved via methods. These properties are:

- setter methods: for setting the value of a property. They start with 'set'. e.g. setName()

- getter methods: for retrieving the value of a property. They start with 'get'. e.g. getName()

3. Properties can be simple types like int or boolean or object types. For object types, the property can be another JavaBean. This allows composition of complex JavaBeans from simpler ones.
4. JavaBeans use introspection to discover properties, setters and getters at runtime using reflection. This makes them flexible and reusable in various contexts.
5. The naming conventions and patterns for properties, setters and getters allow JavaBeans to be configured via external applications or IDE tools by just knowing the property names. The internal implementation details are hidden.
6. JavaBeans follow certain other standards like having a no-arg constructor, handling events etc which make them interoperable. They promote reusable component-based software architecture.

The content summarizes the key points about JavaBeans properties in a formal tone with points and without any emojis or external links. Please let me know if you would like me to modify or expand the answer.

Here is the content in markdown format without any emojis or external links, written in a formal tone with points:

Types of beans in Enterprise Java Bean

1. Session Beans: Used to encapsulate business logic. Two types:

- Stateless: Does not store client state. Scales well and receives client requests.
- Stateful: Stores client state. Suitable for conversations with clients.

2. Message-Driven Beans: Used to process JMS messages asynchronously. They act as listeners and are activated by message arrival.

3. Entity Beans: Used to map objects to relational data. Two types:

- Bean-Managed Persistence: Developer manages persistence.
- Container-Managed Persistence: Container manages persistence.

The content is written in a formal manner like study material without any feelings or friendliness, using points and markdown format as requested. Please let me know if you would like me to modify or expand the content.

Here is the content in Markdown format without any emojis or external links:

Stateful Session bean in Enterprise Java Bean

1. Stateful Session Beans maintain conversational state with the client. This means that the bean retains the client's state across multiple method calls.

2. The state of a stateful session bean is bound to a specific client. As long as the client is connected to the EJB container, the bean's state is kept in memory. If the client terminates, or becomes disconnected, the stateful session bean and its state are removed after a timeout period.
3. Stateful session beans are best suited for applications that require user-specific state to be maintained across multiple client invokes, or in applications that require the bean to hold a complex state relating to the client. Examples include online shopping carts, user sessions etc.
4. The lifecycle of a stateful session bean includes the following states:

- Does Not Exist - When the bean is first created
- Method Ready Pool - After it is created by the container
- In Use - When a client invokes on it
- Passive - When the bean is in the pool but not selected for use

The key points to remember about stateful session beans are:

- They maintain state across method calls
- State is tied to a specific client
- Ideal for applications requiring conversational state
- Uses a bean-instance pool
- Lifecycle states include Does Not Exist, Method Ready Pool, In Use and Passive

Here is the content in formal tone with points and without emojis or external links:

Stateless Session bean in Enterprise Java Bean

1. Stateless Session Beans do not maintain conversational state with the client. This means that Stateless Beans do not store information about a client's session and do not maintain the client's state across method calls.

2. Since Stateless Beans do not store session state, they can support a large number of clients and spread load across multiple server instances. This is useful in performing transactional operations or stateless interactions like retrieving or updating data.

3. A Stateless Session Bean has a bean instance pool so that container provides an instance of the bean to service client requests. When a client finishes with the bean instance, it returns to the pool for reuse. This allows a small number of bean instances to service a large number of clients.

4. Due to no conversational state and bean instance reuse, Stateless Session Beans scale well and provide high performance. This makes them suitable for transactional and stateless operations including interacting with data sources.

5. Since Stateless Beans do not store client state, each method invocation must contain all the information necessary to complete the request and

the bean cannot rely on client state.

This outlines the key characteristics and usage of Stateless Session Beans in Enterprise Java Beans. Stateless Beans are suitable for stateless and transactional interactions and provide high scalability and performance.

Here is the content in markdown format with formal tone and without emojis or external links:

Entity bean in Enterprise Java Bean

1. An Entity bean represents the data (objects) and the behavior (methods) of those objects in the enterprise bean.
2. An entity bean is identified by a primary key. The EJB container uses the primary key to uniquely identify each entity bean instance.
3. The lifecycle of an entity bean has the following states:

- Does not exist: When the bean is not associated with any EJB object
- Pooled: When the bean instance is associated with the pool of available instances
- Method-ready: When the bean instance is enlisted to service a client's method call
- In-use: When the bean instance is executing a client's method call

4. There are two types of entity beans:

- Container-managed persistence (CMP): The EJB container manages the persistent state of the entity bean. The bean class does not contain any database access code.
- Bean-managed persistence (BMP): The entity bean manages its own persistence. The bean class contains the database access code using JDBC API.

5. The deployment descriptor of an entity bean contains the following elements:

- Entity: Defines the entity bean and provides its name.
- Primiry key: Specifies the primiry key of the entity bean and its type.
- Home interface: Specifies the home interface of the entity bean.
- Remote interface: Specifies the remote interface of the entity bean.
- Local interfaces: Specifies the local interfaces of the entity bean.
- Local home interface: Specifies the local home interface of the entity bean.

Does this content meet your requirements? Let me know if you would like me to modify or add anything.

Here is the content in the requested format:

Java Database Connectivity (JDBC)

1. JDBC is an API (Application Programming Interface) that allows Java programs to interact with a relational database.
2. It provides a standard way for Java programmers to access database data from different vendors.
3. The JDBC API includes interfaces and classes that can be used to connect to a database, execute SQL statements, retrieve results etc.
4. The basic steps to access a database through JDBC are:

- Load the JDBC driver: The DriverManager class is used to load and register the driver.
- Make a connection: A Connection object represents the connection to the database. The DriverManager's getConnection() method is used to obtain a Connection.
- Create a statement: A Statement object is used to execute a SQL statement. createStatement() method of the Connection is used to create a Statement.
- Execute a query: The executeQuery() method of the Statement is used to execute a SELECT query and retrieve results.
- Process the results: The ResultSet object contains the results of the query. Next() method of ResultSet is used to iterate over the results.
- Close connections: Finally, the connections, statements and result sets should be closed to release resources.

5. There are 4 types of JDBC drivers:

- Type 1: JDBC-ODBC bridge driver
- Type 2: Native API driver
- Type 3: Network protocol driver
- Type 4: Thin driver

The content is written in markdown format without emojis or external links as requested. The tone is formal and content is written as study material in points. Please let me know if you would like me to modify or add anything.

Here is the formal content in Markdown format without emojis and external links:

Merging Data from Multiple Tables in JDBC

1. Fetch data from multiple tables in separate ResultSets.
2. Create a new ResultSet by appending columns from multiple ResultSets. This new ResultSet will contain merged data from multiple tables.
3. The number of rows in the final ResultSet will be equal to the minimum number of rows among all the input ResultSets.

To merge data from multiple tables, follow the steps below:

1. Fetch data from the first table and store in ResultSet 1.

2. Fetch data from the second table and store in ResultSet 2.
3. Create a new ResultSet by appending columns of ResultSet 1 and 2. The Metadata of the new ResultSet will contain columns from both the tables.
4. Write code to fetch rows from ResultSet 1 and 2 in a loop and add rows to the final merged ResultSet. The loop will run till there are rows in both ResultSet 1 and 2 or in any one of the two. This ensures that the final ResultSet contains minimum number of rows present in the two input ResultSets.
5. Process the merged ResultSet to display or store the data as required.

The code to merge data from two tables may look like:

```
ResultSet rs1 = stmt.executeQuery("SELECT * FROM TABLE1");
ResultSet rs2 = stmt.executeQuery("SELECT * FROM TABLE2");

ResultSetMetaData rsmd1 = rs1.getMetaData();
ResultSetMetaData rsmd2 = rs2.getMetaData();

int columnCount1 = rsmd1.getColumnCount();
int columnCount2 = rsmd2.getColumnCount();
int totalColumns = columnCount1 + columnCount2;

String[] columnNames = new String[totalColumns];
for(int i=1; i<=columnCount1; i++) {
    columnNames[i-1] = rsmd1.getColumnName(i);
}
for(int i=1; i<=columnCount2; i++) {
    columnNames[columnCount1 + i - 1] = rsmd2.getColumnName(i);
}

ResultSet mergedRS = createMergedRS(columnNames, rs1, rs2);
```

Here is the formal content written in Markdown format without any emojis or external links:

Joining in JDBC

1. Inner join: Returns records that have matching values in both tables.
2. Left outer join: Returns all records from the left table, and the matched records from the right table.
3. Right outer join: Returns all records from the right table, and the matched records from the left table.
4. Full outer join: Returns all records when there is a match in either left or right table.

To perform a join in JDBC:

1. Create statements for both tables specifying the join condition in the

WHERE clause.
2. Create a ResultSet for the join statement.
3. Use ResultSet methods such as getString() to retrieve the data from the joined tables.

The key points to keep in mind when joining are:

1. Ensure the JOIN column names are same in both tables.
2. Clearly specify the JOIN type - INNER, LEFT OUTER etc.
3. Use aliases if the column names are ambiguous.
4. Filter the results using WHERE clause if required.

Here is the formal content in Markdown format without any emojis or external links:

Manipulating in JDBC

1. Inserting Records

- Prepare the INSERT query with placeholder parameters
- Prepare a PreparedStatement object from the Connection object
- Set the placeholder parameter values
- Execute the query
- Close resources

2. Updating Records

- Prepare the UPDATE query with SET clause and WHERE clause
- Prepare a PreparedStatement object from the Connection object
- Set the SET clause parameter values and WHERE clause parameter values
- Execute the update query
- Close resources

3. Deleting Records

- Prepare the DELETE query with WHERE clause
- Prepare a PreparedStatement object from the Connection object
- Set the WHERE clause parameter values
- Execute the delete query
- Close resources

The above points describe the steps to perform insert, update and delete operations using JDBC on a database. This can be used as study material to learn and understand JDBC database manipulations for exams.

Here is the content in markdown format with formal tone without any emojis or external links:

Databases with JDBC in JDBC

1. JDBC stands for Java Database Connectivity. It is a Java API that can access any kind of tabular data, especially relational databases.
2. Using JDBC, we can connect to a database, query it, fetch results, update records, or delete records.
3. To use JDBC, we need to follow these steps:

- Import the JDBC package: import java.sql.*
- Load and register the driver: Class.forName("com.mysql.jdbc.Driver")
- Create a connection to the database: conn = DriverManager.getConnection(url, username, password)
- Create a statement to query the database: stmt = conn.createStatement()
- Execute the query: rs = stmt.executeQuery(sql)
- Process the results: while (rs.next()) { // Access column data }

4. We need to download the JDBC driver for the specific database we want to connect to. For example, for MySQL we need mysql-connector-java and for PostgreSQL we need postgresql-jdbc drivers.
5. With JDBC we can use SQL to query and manipulate data in the database. We can parametrize queries to prevent SQL injection.
6. JDBC provides transaction management capabilities to manage atomicity, consistency, isolation, and durability (ACID) of database transactions.

The above points cover the key aspects of working with databases using JDBC in Java. Please let me know if you would like me to elaborate on any of the points or add more details to the content.

Here is the content in markdown format with formal tone and without external links or emojis:

Prepared Statements in JDBC

- Prepared Statements are precompiled SQL statements. The database parses, compiles, and optimizes the SQL once, and then reuses the prepared statement many times. This results in better performance.
- The preparation occurs immediately when you call the prepareStatement() method on the Connection object. The SQL statement is sent to the database for parsing, compilation, and optimization.
- A PreparedStatement object contains a precompiled SQL statement. This statement is parameterized, with parameter markers (?) at appropriate places within the SQL.
- When the PreparedStatement is executed, the parameter markers are replaced by actual values. This replacement of parameter markers with values is known as binding.
- The key benefits of Prepared Statements are: – Better performance: SQL statements are parsed once, and execution plans are reused. – Prevention of SQL injection: The parameter markers ensure that only specified values are inserted into the SQL, eliminating the risk of malicious user

input. – Readability: Prepared Statements separate SQL commands from application variables, making the code easier to read and maintain.

The points highlight the key benefits and steps involved in using Prepared Statements in JDBC for efficient database interactions. The content is written in a formal tone with Markdown formatting and without external links or emojis as specified. Please let me know if you would like me to modify or expand the answer.

Here is the content in Markdown format without any emojis or external links:

Transaction Processing in JDBC

- A database transaction is a unit of work that is performed on a database.
- All steps in a database transaction are executed successfully as a single unit or none are executed at all if any step fails. This is known as ACID transaction properties - Atomicity, Consistency, Isolation, Durability.
- JDBC supports transactions via the java.sql.Connection object. To enable transactions, we need to set the auto-commit mode to false by calling Connection.setAutoCommit(false).
- Once auto-commit mode is off, no SQL statements are committed until we call the method commit explicitly.
- If we want to rollback and undo all changes made in the transaction, we call the rollback method.
- Thus in JDBC, we follow the following steps:

1. Obtain a Connection object
2. Turn off auto-commit mode
3. Perform SQL operations
4. If all operations are successful, call commit
5. If any operation fails, call rollback

- By default, transactions are isolated from each other. This ensures that concurrent executions of transactions results in a system state that would be obtained if the transactions were executed sequentially in some order.
- JDBC Driver Managers manage a pool of connections and transactions can span across multiple connections.
- Savepoints can be used to segment a transaction into smaller points so that if a rollback is required, we can rollback to a savepoint instead of the beginning of the transaction.

Does this look okay? Let me know if you would like me to modify or add anything.

Here is the content in Markdown format without any emojis or external links:

Stored Procedures in JDBC

- A stored procedure is a group of SQL statements that are stored in the database.
- To use stored procedures with JDBC, follow these steps:

1. Create the stored procedure in the database using SQL.
2. Load the driver and establish a connection.
3. Create a CallableStatement object to call the stored procedure.
4. Register the input and output parameters for the stored procedure call.
5. Execute the stored procedure call.
6. Process the results/output parameters.

- The advantages of using stored procedures are:
- Increased performance: The database can optimize the procedures and reuse the execution plans.
- Reduced network traffic: Only the call to the procedure needs to be sent to the database, rather than the full SQL statements.
- Encapsulation: The logic is stored in one place and can be maintained easily.
- Reusability: The procedures can be reused by multiple applications.
- Security: Permissions can be granted on a procedure-by-procedure basis.

The above content is written in a formal tone with points and without any emojis or external links as per the given instructions. Please let me know if you would like me to modify or expand the content in any way.

Here is the content in markdown format with formal tone and without emojis:

Unit 5 - Servlets

1. Servlets are Java programs that run on a web server. They are used to handle requests and generate responses to the client.
2. The steps to create a servlet are:

- Extend the HttpServlet class
- Override the doGet/doPost methods to handle GET and POST requests respectively
- Configure servlet mapping in web.xml to map an URL to the servlet

3. The life cycle of a servlet involves the following stages:

- Loading: The servlet is loaded into memory when the web container starts up
- Initialization: The init() method is called to initialize the servlet
- Request handling: The service() method is called to handle client requests
- Destruction: The destroy() method is called when the servlet is unloaded from memory

4. Servlets can maintain state between client requests using techniques like session tracking and request dispatching. Sessions can be tracked using cookies, URL rewriting or the HttpSession object.

5. Advantages of servlets are:

- They are robust, fast and efficient
- They can access full Java APIs
- They are cost effective and platform independent
- They support multithreading and can handle large loads

The above content summarizes the key points about servlets in a formal tone with headings and points. External links are avoided and all content is written manually as instructed. Please let me know if you would like me to modify or add any other points.

Here is the content in markdown format without any emojis or external links:

Servlet Overview and Architecture in Servlets

1. Servlet is a Java programming language class used to extend the capabilities of servers that host applications accessed by way of a request-response programming model.
2. Servlets run on a Java Virtual Machine (JVM) and can respond to requests either from browsers or remote programs (e.g. applets, XMLHttpRequests etc.)
3. The lifecycle of a servlet consists of the following phases:

- Loading: The servlet is loaded into memory.
- Initialization: The init() method is invoked. Any one-time initialization occurs here.
- Ready: The servlet waits to service client requests.
- Processing request: The service() method (or doGet() or doPost() methods) are invoked to process client requests.
- Destroy: The destroy() method is called when the servlet is taken out of service. Resources can be freed here.

4. The key components of the servlet architecture are:

- Servlet: Java class that extends the capabilities of a server.
- Servlet Container: Manages the lifecycle of servlets, maps a URL to a servlet, and ensures that the servlet is run securely.
- Servlet Config: An object that contains information on initialization parameters and servlet context.
- Servlet Context: Defines a set of methods that a servlet uses to communicate with its servlet container.

The content is written in points and in a formal tone without any emojis or external links as instructed. Please let me know if you would like me to modify or add anything.

Here is the content in markdown format without any emojis or external links:

Interface Servlet and the Servlet Life Cycle in Servlets

1. Interface Servlet: The javax.servlet.Servlet interface is implemented by servlets and defines the basic methods that all servlets must implement. These methods include:

- init(): Initializes the servlet and is called only once.
- service(): Processes requests from clients and is called on each request.
- destroy(): Terminates the servlet and is called only once when the servlet is destroyed.

2. Servlet Life Cycle: The life cycle of a servlet describes the various stages that a servlet goes through from when it is instantiated until it is destroyed. The stages are:

- Loading and Instantiation: When the servlet container starts up, it loads and instantiates all servlets defined in the web application deployment descriptor.
- Initialization: The servlet's init() method is called, and the servlet is initialized with any initialization parameters from the deployment descriptor.
- Request Handling: For each client request, the service() method is called. The request and response objects are passed as parameters. Multiple requests can be handled concurrently in a multithreaded servlet container.
- Termination or Destroy: The destroy() method is called when the servlet is removed from service. This happens when the servlet container is shut down or the servlet is reloaded or undeployed. The servlet releases any resources such as database connections that it is holding.

The content is written in points and in a formal tone without any emojis or external links as per the instructions. Please let me know if you would like me to modify or add any additional points.

Here is the formal content in markdown format without any emojis or external links:

Handling HTTP get Requests in Servlets

1. Create a Servlet class by extending HttpServlet class
2. Override the doGet method of HttpServlet class
3. In doGet method, get the request parameters using request.getParameter() method
4. Based on the request parameters, execute the required business logic
5. Set the response content type using response.setContentType() method
6. Write the response in the output stream using response.getWriter().print() method
7. Compile the Servlet class and deploy it in the Web container
8. In the browser, hit the URL of the deployed servlet along with query parameters
9. The servlet will handle the GET request and provide response accordingly

The doGet() method of HttpServlet class gets invoked whenever there is a GET request received from the client. We can access the request parameters, perform required processing and send back the response to the client using this method. The response can be an HTML page, JSON data or XML data based on the requirement.

Hope this content serves the purpose of providing formal study material on the topic of handling HTTP GET requests in Servlets. Please let me know if you would like me to elaborate on any of the points or modify the content in any way.

Here is the content in markdown format:

Handling HTTP post Requests in Servlets

1. Retrieve the request parameters:

- Use `request.getParameter()` method to retrieve the form parameters sent in the post request.
- The parameters are retrieved as strings.
- In case of multiple values for a parameter, `request.getParameterValues()` can be used which returns an array of string values.

2. Read the request body:

- For large requests, the parameters are sent in the request body and not in the URL.
- To read the request body, we need to get the body as an InputStream using `request.getInputStream()` or `request.getReader()`.
- The InputStream can then be used to read the data in the required format (eg. XML, JSON etc.).

3. Handle the data in the request:

- The retrieved parameters and request body data can then be processed as required.
- This may involve storing the data in a database, performing some calculations on it, sending emails etc. based on the application logic.

4. Send response:

- Send an appropriate response to the client indicating the success or failure of the post request processing.
- This can be a simple redirection to a page or can include a JSON/XML response if the request was an AJAX request.

The points are written in a formal tone without any feelings or emojis and contain only written content (no images/links). The content summarizes how to handle HTTP post requests in servlets. Please let me know if you would like me to modify or expand the answer.

Here is the content in markdown format without any emojis or external links:

Redirecting Requests to Other Resources in Servlets

1. Using response.sendRedirect(): This method redirects the client to a new URL. It takes the new URL as a parameter. For example, `response.sendRedirect("newURL");`
2. Using response.setStatus() and response.setHeader(): We can set the status code to 302 (FOUND) and add a Location header with the new URL. For example, `response.setStatus(302); response.setHeader("Location", "newURL");`
3. Using RequestDispatcher: We can forward the request to another servlet/JSP using `RequestDispatcher.forward()` method. We get the `RequestDispatcher` using `request.getRequestDispatcher()`. For example, `rd = request.getRequestDispatcher("url"); rd.forward(request, response);`

Advantages of redirect over forwarding: - The address bar will display the redirected resource's URL, making the redirection more obvious to the user. - The redirected resource is loaded as a new resource, thus avoiding potential issues with the forwarded resource. - Search engines consider redirects as a ranking signal.

I have written the content in points and in a formal tone without any emoji or external links as instructed. Please let me know if you would like me to modify or expand the content.

Here is the content in markdown format without any emojis or external links, written in points and in a formal tone:

Session Tracking in Servlets

1. HttpSession: Servlets can use the HttpSession object to track sessions. When a client accesses a servlet the first time, a session is created and a session ID is sent to the client. In subsequent requests, the client passes on this session ID which is used by the servlets to identify the session. Servlets can store attributes/objects in the session and retrieve them as long as the session is valid.
2. URL rewriting: The session ID can be embedded in the URLs accessed by the client. This is called URL rewriting. The embedded session ID is used by the servlets to track the client's session. The disadvantages are that the actual URL is obscured and the length of the URL increases with a long session ID.
3. Hidden form fields: The session ID can be stored in a hidden form field and sent to the servlets on form submission. This method faces issues if the client disables cookies as the session ID cannot be stored.
4. Cookies: Cookies are the most commonly used method to track sessions in servlets. A cookie containing the session ID is sent to the client by the

servlet and stored by the browser. The cookie is sent back to the servlets with every request and enables the servlets to identify the client session.

The points are written in a formal tone with no emojis or external links as per your instructions. Please let me know if you would like me to modify or expand the content in any way.

Here is the content in markdown format with formal tone and without emojis:

Cookies in Servlets

1. Cookies are small pieces of data sent from a web server to a web browser. The browser stores the cookies and sends them back to the server with every request.
2. This allows the server to keep track of the user over multiple pages or sessions. For example, a cookie can store a session ID to identify a user in a session management mechanism.
3. To set a cookie in a servlet, use the setHeader() method of the HttpServletResponse object. You need to specify a name and value for the cookie, as well as optionally an expiration date and path.
4. For example: response.setHeader("Cookie", "name=value");

This will create a session cookie that expires when the browser is closed.

response.setHeader("Set-Cookie", "name=value; expires=Wed, 09 Jun 2021 10:18:14 GMT; path=/");

This will create a persistent cookie that expires on a specific date. The path indicates what path the cookie is valid for.

5. To retrieve cookies in a servlet, use the getHeaders() method of the HttpServletRequest object and look for "Cookie" headers. You'll need to parse the cookie header string to get the values of the individual cookies.

For example: String cookie = request.getHeader("Cookie"); // Parse cookie…

This can be useful for session management, personalization, and tracking users on a website. However, cookies do have privacy implications that must be considered.

Here is the content in Markdown format without any emojis or external links:

Session Tracking with Http Session in Servlets

1. HttpSession object is used to track a user across multiple requests.
2. When a user visits a web application for the first time, the server creates a HttpSession and assigns a unique session ID to it. This session ID is sent back to the browser via cookie or URL rewriting.
3. On subsequent requests, the browser sends back this session ID. The server uses this ID to retrieve the associated HttpSession and thereby track the user.

4. Data can be stored in the HttpSession as attributes. This data persists as long as the session is valid.
5. The validity of a session can be configured using following ways:

- Inactive Interval: If a session is not accessed for a configured time interval, it can be invalidated.
- Max Inactive Interval: A session will be forcibly invalidated after a configured maximum inactive interval.
- Manual Invalidation: A session can be manually invalidated by calling invalidate() method on HttpSession.

6. Session data can be used to store user specific information like login details, shopping cart data, etc and used across multiple requests.
7. However, excessive use of sessions can impact performance. Hence, sessions should only be used when required and data stored in sessions should be minimized.

The content summarizes key points about HttpSession and session tracking in Servlets in a formal tone with points and without any feelings or emojis as instructed. Please let me know if you would like me to modify or expand the content.

Here is the content in markdown format without any emojis or external links:

Java Server Pages (JSP) in Servlets

1. JSP is a technology that allows you to generate dynamic web pages that contain HTML, XML, and JSP tags.
2. JSP pages are compiled into servlets and execute on the server to generate web pages.
3. A JSP page contains two sections:

- A static template section: This contains the HTML or XML code along with JSP tags.
- A dynamic section: This contains Java code that is executed on the server and the output is sent to the client.

4. The JSP tags that start with <%@ include directives for importing packages and custom tags, and configuring the page.
5. The JSP tags that start with <% include scriptlet tags for Java code, and expression tags for outputting values.
6. A JSP container handles the conversion of JSP pages into servlets, manages their lifecycle, and provides additional services.
7. You can also include JavaBeans components (reusable software components) in a JSP page to keep the presentation layer separate from the application logic.
8. Advantages of JSP:

- It separates the presentation layer (HTML/XML) from the application logic (Java code).
- It is easy to make changes to the presentation layer.
- It provides more functionality than servlets and is easier to use.
- The JSP engine takes care of converting JSP pages into servlets.

The content is written in bullet points and formal tone as instructed without any emojis or external links. Please let me know if you would like me to modify or add any other points to the content.

Here is the content in formal tone without emojis or external links in markdown format:

Introduction to JSP in Servlets

1. JSP stands for JavaServer Pages. It is a technology that allows us to mix Java code with HTML to create dynamic web pages.
2. JSP pages use special JSP tags, which are interpreted by the JSP engine to generate appropriate Java servlet. The servlet is then compiled and executed to generate the final web page.
3. JSP pages have a .jsp extension. They can contain static template data, HTML, Java code and JSP tags.
4. The initial JSP page is compiled into a Java servlet by the JSP engine. Whenever the JSP page is requested, the servlet is executed and sends the response to the client. This makes JSP pages platform independent.
5. Some of the JSP tags are:

- <% %> - For embedding Java code
- <%= %> - For embedding Java expressions
- - For including another JSP or HTML page
- - For using JavaBeans components
- - For forwarding request to another page

6. JSP technology allows us to separate the presentation layer from the business logic. The HTML and JSP tags constitute the presentation layer, whereas the Java code constitutes the business logic. This makes the web application easy to maintain.
7. JSP technology can be used along with servlets to build powerful and dynamic web applications. Servlets handle the business logic and JSP pages generate the UI.

Here is the content in Markdown format without any emojis or external links:

Java Server Pages Overview in Servlets

1. Java Server Pages (JSP) is a technology that allows you to mix regular, static HTML with dynamically-generated Java code called scriptlets.
2. The code is executed on the server and its output is sent to the client as a plain HTML document.

3. JSPs use a .jsp file extension and are compiled into servlets, which are Java classes that handle requests and generate responses.
4. The compilation happens automatically the first time the JSP is accessed. After that, the servlet is used to handle requests to the JSP.
5. JSPs can use various JSP elements to insert dynamic content and control the page logic:

- Scriptlets: blocks of Java code between <% %>
- Expressions: insertion of values into the page using <%= %>
- Declarations: declaration of variables or methods using <%! %>
- Directives: messages to the JSP engine using <%@ %>

6. JSPs provide an easy way to create web pages that display dynamic content. However, for more complex applications, it is recommended to use Java servlets or JSP custom tags instead of scriptlets.

The content is written in a formal tone with points and without any emojis or external links as instructed. Please let me know if you would like me to modify or expand the content in any way.

Here is the content in Markdown format without any emojis or external links, written in a formal tone with points:

A First Java Server Page Example in Servlets

1. A Java Server Page (JSP) is a text-based document that contains two types of elements: static elements and dynamic elements.
2. Static elements are the regular HTML tags and text that display static content to the user.
3. Dynamic elements are JSP tags such as <% %>, <%= %>, and <%! %> that are processed by the JSP engine to generate dynamic content.
4. A JSP is translated into a servlet by the JSP engine, compiled, and executed on the server to generate dynamic web pages.
5. The following is a basic JSP example that displays “Hello World”:

<%@ page language=“java” contentType=“text/html; charset=UTF-8” pageEncoding=“UTF-8”%>

Hello World!

6. The JSP page begins with a page directive that specifies the scripting language as Java, and the character encoding. The remaining lines contain regular HTML that displays a heading.
7. When the JSP is translated into a servlet, the JSP engine inserts appropriate Java code that results in the text “Hello World!” being displayed in the web page.

Here is the content written in markdown format with formal tone and without any emojis or external links:

Implicit Objects in Servlets

- `request`: It is an object of `HttpServletRequest` class and contains information about the HTTP request sent by the client. Data such as request parameters, headers, request URI, etc. can be obtained using this object.
- `response`: It is an object of `HttpServletResponse` class and is used to create HTTP response to be sent to the client. We can specify response status code, add headers, get output stream, etc. using this object.
- `out`: It is a `PrintWriter` object used to write the response to the client. It basically wraps the `getWriter()` method of the `response` object.
- `application`: It is an object of `ServletContext` interface and refers to the servlet context of the web application deployed on the server. It can be used to store attributes at application level and obtain MIME types, resource paths, etc.
- `session`: It is an object of `HttpSession` interface and represents the user session. Attributes can be stored and retrieved using session for a particular user. Session tracking is enabled using this object.
- `pageContext`: It is an object of `PageContext` class and is available only to JSP pages, not servlets. It encapsulates different implicit objects and provides additional functionality like inclusion of other resources.

The above implicit objects can be directly used in servlets and JSP pages without explicitly declaring them. They provide information regarding the request, response, application, session, etc. which reduces the amount of boilerplate code required to be written.

Here is the formal content on the topic "Scripting in Servlets" in Markdown format without any emojis or external links:

Scripting in Servlets

1. Servlets can include scriptlets - small segments of Java code - that is executed to generate dynamic content.
2. Scriptlets are included between <% %> tags.
3. Example:
   <% int i = 0; while (i < 5) { %>
   Count: <%= i %>
   <% i++; } %>
4. The above scriptlet will generate 5 paragraphs with the count from 0 to 4.
5. The <%= %> tags are used to output expressions. The equal sign (=) differentiates an expression from a scriptlet.
6. Scriptlets allow access to the full features of the Java programming language to control the content and flow of a Servlet.
7. However, the use of scriptlets should be minimized for:

- Maintainability - interspersing Java code with HTML makes the Servlet harder to read and maintain

- Separation of Concerns - it is better to separate the presentation logic (HTML) from the business logic (Java)

8. It is preferable to use JSP tags and expressions to generate dynamic content where possible instead of scriptlets.

Here is the content in markdown format with formal tone and without any emojis or external links:

Standard Actions in Servlets

1. doGet(): Called by the server (via the service() method) to allow a servlet to handle a GET request. This method is used when a browser issues an HTTP GET request to the servlet.
2. doPost(): Called by the server (via the service() method) to allow a servlet to handle a POST request. This method is used when a browser issues an HTTP POST request to the servlet.
3. init(): Called by the server (via the init() method) to initialize a servlet. This method is executed only once when a servlet is first loaded. It's used to perform any one-time initialization.
4. destroy(): Called by the server (via the destroy() method) to allow a servlet to perform any clean-up before it is unloaded. This method is executed only once when the servlet is about to be unloaded. It's used to release any resources that the servlet may have occupied.

The service() method of the HttpServlet class dispatches the HTTP requests to the appropriate doXXX() methods. A servlet container calls the service() method for every request and passes in an HttpServletRequest and an HttpServletResponse object.

The doGet() and doPost() methods handle GET and POST requests respectively. The programmer should override these methods to specify the request handling behavior of their servlet for these requests.

The init() and destroy() methods are called by the servlet container to manage the lifecycle of the servlet. The programmer can perform any initialization tasks or cleanup jobs in these methods respectively.

Here is the content in markdown format with formal tone and without any emojis or external links:

Directives in Servlets

- `<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>` - This directive specifies the scripting language of the servlet, content type and the encoding format. It is the first line of any servlet and contains all the metadata about the servlet.
- `<%@ include file="common.jsp"%>` - This is an include directive which includes the content of the specified file (common.jsp in this case) at the

location of the directive. This is useful to avoid repetition and promote reuse of code.
- `<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>` - This is a taglib directive which declares a tag library with a given URI and prefix. The prefix is then used to use the tags from the library in the JSP page.

The directives are processed by the JSP engine even before the servlet is compiled. They provide important information to the JSP engine to process the page. The directives start with `<%@` and end with `%>`.

Here is the content in markdown format with formal tone and without emojis:

Custom Tag Libraries in Servlets

1. Custom tag libraries in Servlets allow us to create our own tags which can be used to encapsulate common functionality in JSP pages.
2. We can group related tags in a library and reuse them across multiple JSP pages. This improves maintainability of the code.
3. To create a custom tag library, we need to follow these steps:

- Create a TLD (Tag Library Descriptor) file which defines the tags in the library along with their attributes.
- Create Java classes which implement the tag logic. These classes must extend javax.servlet.jsp.tagext.TagSupport class.
- Package the TLD file and the Java classes into a JAR file.
- Deploy the JAR file in the web application's /WEB-INF/lib directory.
- Use the tags in JSP pages and provide required attributes to the tags.

4. The benefits of custom tag libraries are -

- Code reusability - Common functionality can be encapsulated in tags and reused.
- Maintainability - Changes need to be made at only one place if tags are reused.
- Readability - JSP pages using tags are more readable than having scriptlets.
- Separation of concerns - Presentation logic can be separated from the business logic using custom tags.